ENTERPRISE CLOUD-NATIVE SUITE (BILINGUAL / ÇİFT DİLLİ)

Aptala Anlatır Gibi Kubernetes & GitOps

Explain Like I'm 5: K8s, OpenShift, k9s, Operators, Flux & GitHub Actions

10 Bölüm • 60 Ders • Java Spring Boot Odaklı Uçtan Uca Kurumsal Rehber

---

🐋 Docker & K8s

Container mimarisi, Pod'lar, Servisler, Ingress

🛡️ OpenShift & k9s

Kurumsal OCP, SCC, Terminal Matrix Kokpiti

⚡ FluxCD & CI/CD

GitOps, Operators, GitHub Actions Pipeline

Yayın Sürümü: 1.0.0 • Java 21 & Spring Boot 3.x Uyumlu • 2026

Önsöz ve Başlarken / Preface & Getting Started

🇹🇷 Bu Eğitim Hakkında

Bu kitap; Kubernetes, Red Hat OpenShift, k9s, Kubernetes Operators, FluxCD, Docker ve GitHub Actions gibi modern bulut-yerel (Cloud-Native) dünyasını **bir aptala anlatır gibi** en sade, en eğlenceli ve en akılda kalıcı metaforlarla öğretmek için tasarlanmıştır.

Her ders; gerçek dünyadan benzetmeler, teknik derinlik, üretime hazır Java Spring Boot 3.x kod örnekleri ve acemi tuzakları içerir. 60 dersin sonunda tek satır koddan canlı kümede çalışan zırhlı mikroservislere kadar tam bir kurumsal döngüyü bizzat yöneteceksiniz!

🇬🇧 About This Handbook

This handbook is engineered to demystify Kubernetes, Red Hat OpenShift, k9s, Kubernetes Operators, FluxCD, Docker, and GitHub Actions using crystal-clear **Explain Like I'm 5 (ELI5)** metaphors and zero gatekeeping.

Every lesson pairs side-by-side Turkish and English explanations, real-world analogies, enterprise-grade Java Spring Boot 3.x manifests, and battle-tested SRE pro-tips. By Lesson 60, you will confidently architect and operate complete GitOps continuous delivery fleets!

İçindekiler / Table of Contents

🇹🇷 Bu Eğitim Hakkında 2

🇬🇧 About This Handbook 2

BÖLÜM 1 / MODULE 1

Konteyner Dünyası & Docker Temelleri

Container World & Docker Fundamentals

Yazılım dünyasının standart kargo konteynerleri: Sanal makinelerden Docker'a, multi-stage Spring Boot derlemelerine ve ağ yönetimine sıfırdan adım atış.

The standard shipping containers of software: From VMs to Docker, multi-stage Spring Boot builds, volumes, and networks from scratch.

1

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

"Ama Benim Bilgisayarımda Çalışıyordu!" – Sanal Makineler vs Konteynerler

"It Worked on My Machine!" – Virtual Machines vs Containers

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Eskiden bir tüccar kumaş, diğeri zeytinyağı fıçısı, öbürü gevşek buğday çuvalı taşırdı. Gemiyi yüklemek günler sürer, çuvallar patlar, her şey birbirine karışırdı! 1956'da standart çelik kargo konteyneri icat edildi. Artık gemi vincinin içinde ne olduğunu bilmesine gerek yok; standart kutuyu alır, koyar! Docker da yazılım dünyasının standart çelik kutusudur. Sanal Makine (VM) koca bir evi temeliyle taşımak gibidir; Docker Konteyneri ise sadece eşyalarınızı bavula koyup taşımak gibidir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Back in the 1950s, dockers loaded odd wooden barrels, loose bags of grain, and fragile crates by hand. Cargo took days to load and frequently broke. In 1956, the standardized steel shipping container changed human trade forever! Cranes didn't care what was inside—only that it fit the standard locks. Docker is that exact shipping container for software. A Virtual Machine (VM) is like transporting an entire house including foundations; a Container is packing just your suitcase with only the essentials!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Sanal Makineler (Hypervisor Type 1/2) her uygulama için ayrı bir misafir işletim sistemi (Guest OS), ayrı sanal donanım ve kendi çekirdeğini (Kernel) ayağa kaldırır. Bu da gigabaytlarca RAM ve dakikalar süren açılış süresi demektir. Docker konteynerleri ise doğrudan ana makinenin Linux çekirdeğini paylaşır. İzolasyon iki temel Linux çekirdek özelliğiyle sağlanır: 1) Namespaces (Konteynerin sadece kendi süreçlerini, ağını ve dosya sistemini görmesini sağlar; sanal göz bağıdır), 2) Cgroups (Control Groups: CPU ve RAM kotasını sınırlar).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Virtual Machines (Type 1/2 Hypervisors) virtualize physical hardware. Every VM runs a complete Guest OS kernel, consuming gigabytes of memory and taking minutes to boot. Docker containers share the host Linux kernel. Isolation is achieved via two core Linux kernel primitives: 1) Namespaces (isolate PID, Mount, Net, IPC, UTS, User so processes think they own the machine), and 2) Cgroups (Control Groups, metering and limiting CPU, RAM, and IOPS).

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Doğrudan sunucuda çalıştırmak (Kötü Yöntem):
# Java 17 mi kurulu, 21 mi? Hangi glibc sürümü var? Kimse bilmez!
java -jar target/spring-boot-k8s-eli5-1.0.0.jar

# Docker ile standart çelik konteynerde çalıştırmak (Modern Yöntem):
docker run -d -p 8080:8080 --name my-spring-coffee ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Konteynerler tam anlamıyla bir sanal makine değildir! Ana işletim sisteminin çekirdeğini (kernel) kullandıkları için, Linux tabanlı bir konteyner doğrudan saf Windows çekirdeğinde çalışmaz (arka planda WSL2 veya Hyper-V Linux VM gerekir).

🇬🇧 WATCH OUT

Containers are NOT lightweight virtual machines! Because they share the host kernel, Linux containers on Windows/macOS secretly run inside a thin Linux VM (WSL2 / Apple Hypervisor).

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Konteynerler Guest OS yükünü ortadan kaldırarak saniyeler içinde ayağa kalkar.
- Linux Namespaces görünürlüğü, Cgroups ise CPU/Bellek limitlerini kontrol eder.
- Java uygulaması her işletim sisteminde aynı JRE ve bağımlılıklarla garanti çalışır.

- Containers eliminate Guest OS overhead, starting in milliseconds.
- Namespaces provide visibility boundaries; Cgroups enforce resource limits.
- Guarantees identical Java runtime behavior across laptops, staging, and production.

2

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Docker Mimarisi: İmaj (Image), Konteyner (Container) ve Registry

Docker Architecture: Images, Containers, and Registries

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Anneannenizin meşhur elmalı pasta tarifi defterde yazılıdır; buna 'İmaj' (Image) denir. Bu tariften fırında pişirdiğiniz, yiyebileceğiniz sıcacık pasta ise 'Konteyner'dir (Container). Tek bir tariften 100 tane özdeş pasta pişirebilirsiniz. Yemek tarifi siteleri (Docker Hub, GitHub Packages) ise 'Registry'dir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Your grandma's apple pie recipe written in a cookbook is the 'Docker Image'. The actual warm, edible pie baked in the oven is the 'Container'. You can bake 100 identical pies from that single recipe. The online culinary recipe library is the 'Registry' (Docker Hub, Quay, GHCR)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker Image, salt-okunur (read-only) katmanlardan (layers) oluşan bir OCI (Open Container Initiative) paketidir. Union File System (UnionFS/Overlay2) sayesinde her katman bir öncekinin üzerine bindirilir. Bir konteyner başlattığınızda, bu salt-okunur katmanların en tepesine incecik bir 'Yazılabilir Katman' (Writable Container Layer) eklenir. Dosya silindiğinde veya değiştirildiğinde 'Copy-on-Write' (CoW) mekanizması devreye girer.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A Docker Image is an OCI-compliant package consisting of stacked read-only layers. Using an Overlay2 Union File System, each instruction in a Dockerfile generates an immutable layer. When a container runs, Docker attaches a thin writable layer on top. Any file modification uses the Copy-on-Write (CoW) strategy without mutating the underlying image.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# 1. Uzak depodan (Registry) imaj çek
docker pull eclipse-temurin:21-jre-alpine

# 2. İmajları listele
docker images

# 3. İmajdan çalışan bir konteyner üret (Tariften pasta pişir!)
docker run -d --name coffee-service -p 8080:8080 ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0

# 4. Çalışan konteynerleri denetle
docker ps
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Konteyneri durdurup sildiğinizde (docker rm), o en üstteki yazılabilir katmandaki tüm veriler (log-lar, geçici dosyalar) anında yok olur! Kalıcı veri için mutlaka 'Volume' kullanılmalıdır.

🇬🇧 WATCH OUT

When you destroy a container (`docker rm`), its writable top layer is destroyed forever! Never store production databases or uploads inside the container's ephemeral writable layer.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- İmaj değişmez (immutable) bir şablondur; Konteyner onun canlı örneğidir.
- Overlay2 dosya sistemi katmanları paylaşarak disk ve indirme tasarrufu sağlar.
- Registry'ler imajların güvenli dağıtıldığı global kütüphanelerdir.

- An Image is an immutable template; a Container is its running process instance.
- Overlay2 shares identical layers across images to save bandwidth and disk.
- Registries are centralized libraries for sharing versioned container images.

3

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Java Spring Boot Uygulamasını Paketleme: Multi-Stage Build & Layered JAR

Packaging Java Spring Boot: Multi-Stage Builds & Layered JARs

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kocaman un çuvallarını, mikseri ve kirli tencereleri salondaki yemek masasına koymazsınız! Mutfakta yemeği pişirir (Maven ile derleme), salona garsonla sadece şık porselen tabaktaki yemeği yollarsınız (JRE). Multi-stage build tam olarak budur: Derleme araçlarını çöpe at, sadece çalışan Java kodunu son imaja koy!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't serve your dinner guests by wheeling 50kg sacks of flour and dirty mixing bowls into the dining room! You cook in the kitchen (Maven build) and carry only the clean porcelain plate to the table (JRE). Multi-stage builds discard the 800MB compiler tools and package only the lean 50MB runtime!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Spring Boot 2.3+ ile gelen 'Layered JAR' mimarisi, geleneksel tek parça (fat/uber) JAR'ları 4 mantıksal katmana böler: 1) dependencies (nadiren değişir), 2) spring-boot-loader, 3) snapshot-dependencies, 4) application (sizin kodunuz; her commit'te değişir). Dockerfile'da `java -Djarmode=layertools -jar app.jar extract` çalıştırarak bu katmanları ayırırız. Böylece 1 satır Java kodu değiştirdiğinizde Docker sadece 1MB'lık application katmanını yeniden derler; 150MB'lık kütüphaneleri baştan indirmez!

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Spring Boot layered JARs separate fat JARs into 4 distinct layers: 1) dependencies, 2) spring-boot-loader, 3) snapshot-dependencies, 4) application code. By running `java -Djarmode=layertools -jar app.jar extract`, Docker caches the 150MB third-party libraries indefinitely. When you change one Java class, Docker only invalidates and pushes the 2MB application layer!

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Dockerfile'da Katmanları Çıkarma ve Kopyalama:
FROM eclipse-temurin:21-jre-alpine AS extractor
WORKDIR /build
COPY target/spring-boot-k8s-eli5-1.0.0.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract

FROM eclipse-temurin:21-jre-alpine AS runner
WORKDIR /app
COPY --from=extractor /build/dependencies/ ./
COPY --from=extractor /build/spring-boot-loader/ ./
COPY --from=extractor /build/snapshot-dependencies/ ./
COPY --from=extractor /build/application/ ./
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Multi-stage kullanmazsanız, JDK derleyicisi ve Maven yerel deposu (/.m2) imajın içinde kalır; imaj boyutunuz 250MB yerine 1.2GB olur ve güvenlik açıkları tavan yapar!

🇬🇧 WATCH OUT

Without multi-stage builds, your JDK compiler, git, and /.m2 cache leak into production, inflating your image from 200MB to 1.5GB with huge security risks.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Multi-stage build ile derleme ortamı ve çalışma ortamı tamamen ayrılır.
- Spring Boot Layered JAR ile Docker derleme süreleri %90 hızlanır.
- Konteyner imajı küçüldükçe ağ transferi hızlanır, güvenlik açıkları azalır.

- Multi-stage builds decouple build tools from the minimal runtime image.
- Layered JAR caching reduces image build and push times by up to 90%.
- Smaller images transfer faster and significantly reduce attack surface.

4

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Docker Veri Saklama ve Ağ: Volumes ve Bridge Networks

Docker Storage & Networking: Volumes and Bridge Networks

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Konteyner bir otel odası gibidir. Sabah check-out yaptığınızda temizlik görevlisi gelir ve odadaki her şeyi çöpe atar! Kalıcı eşyalarınızı otelin kiralık çelik kasasında (Volume) saklamalısınız. Odalar arası telefon hattı ise Docker Network'tür; oda numarasını (konteyner adını) çevirip doğrudan konuşursunuz.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Containers are like luxury hotel rooms. When you check out, the maid throws everything in the room into the incinerator! To keep your laptop and passport safe, you store them in the hotel's secure basement locker (Docker Volume). The intercom phone between rooms is the Docker Network; just dial the room name to talk.

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker'da üç temel veri saklama yöntemi vardır: 1) Named Volumes (Docker tarafından `/var/lib/docker/volumes` altında yönetilir; en yüksek performans ve taşınabilirlik), 2) Bind Mounts (Host makinedeki doğrudan bir klasörü `/Users/.../data` konteyner içine bağlar; geliştirme anında kod senkronizasyonu için idealdir), 3) tmpfs (Sadece RAM'de saklanır). Ağ tarafında ise varsayılan `bridge` ağı, konteynerler arasında yerleşik DNS çözünürlüğü sunar. Konteynerler birbirine IP ile değil isimleriyle (`postgres-db:5432`) erişir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Storage strategies: 1) Named Volumes (managed under `/var/lib/docker/volumes`, isolated from host OS quirks, best performance), 2) Bind Mounts (mounts arbitrary host paths, ideal for local dev live-reload), 3) tmpfs (stored strictly in host memory). On the networking side, user-defined bridge networks provide automatic embedded DNS resolution between containers by container name.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# 1. Özel bir bridge ağı oluştur
docker network create coffee-net

# 2. Kalıcı bir veri diski (Volume) oluştur
docker volume create postgres-data

# 3. PostgreSQL'i ağa ve diske bağlayarak başlat
docker run -d --name postgres-db --network coffee-net \
  -v postgres-data:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=coffeedb postgres:16-alpine

# 4. Spring Boot'u aynı ağa bağla (IP yerine doğrudan 'postgres-db' adını kullanır!)
docker run -d --name coffee-api --network coffee-net -p 8080:8080 \
  -e SPRING_DATASOURCE_URL=jdbc:postgresql://postgres-db:5432/coffeedb \
  ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Varsayılan (default) bridge ağında konteyner isimleriyle otomatik DNS çözümlemesi çalışmaz! İsimle haberleşme için mutlaka `docker network create` ile kendi özel ağınızı oluşturmalısınız.

🇬🇧 WATCH OUT

The default legacy docker0 bridge does NOT provide automatic DNS resolution by container name! Always create a user-defined bridge network.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Named Volumes konteyner silinse bile veritabanı verilerini korur.
- Named volumes persist state even when containers are stopped and destroyed.
- Docker yerel DNS'i sayesinde konteynerler IP değişse bile birbirini bulur.
- Docker embedded DNS resolves container names dynamically without static IPs.
- Veritabanı ve Spring Boot uygulamalarını aynı özel ağa bağlamak şarttır.
- Co-locating apps and databases on dedicated networks isolates traffic securely.

5

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Docker Compose ile Çoklu Servis: Spring Boot + PostgreSQL Çiftliği

Multi-Service Stacks with Docker Compose: Spring Boot + PostgreSQL

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Restoranda tek başına yemek yapmak kolaydır. Ama aynı anda ekmek fırınını, et ızgarasını ve tatlı reyonunu senkronize başlatmak için bir şefe ve nota kağıdına ihtiyacınız vardır. Docker Compose o nota kağıdıdır (`docker-compose.yml`). Tek bir tuşla (`docker compose up`) tüm orkestra kusursuz sırada sahneye çıkar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Playing a solo flute is easy. Conducting a 20-piece symphony requires sheet music! Docker Compose is the master sheet music (`docker-compose.yml`). With a single command (`docker compose up`), your database, Redis cache, and Spring Boot backend spawn in perfect harmony!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker Compose bildirici (declarative) bir YAML dosyasıyla servisleri, ağları ve diskleri tek bir çatı altında tanımlar. `depends_on` mekanizması başlatma sırasını belirler; `healthcheck` bloğu ile birleştirildiğinde Spring Boot servisinin PostgreSQL tamamen 'ready' olana kadar beklemesi sağlanabilir. Ortam değişkenleri (`environment`), port haritalamaları (`ports`) ve secret dosyaları tek merkezden yönetilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Docker Compose declaratively defines multi-container applications. Using `depends_on` with `condition: service_healthy`, Compose ensures your Spring Boot service pauses booting until PostgreSQL is genuinely accepting TCP connections, eliminating startup connection crashes.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# docker-compose.yml
services:
  postgres-db:
    image: postgres:16-alpine
    container_name: postgres-db
    environment:
      POSTGRES_DB: coffeedb
      POSTGRES_USER: coffeeuser
      POSTGRES_PASSWORD: secretpassword
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U coffeeuser -d coffeedb"]
      interval: 5s
      timeout: 5s
      retries: 5

  coffee-app:
    build: .
    container_name: coffee-app
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-db:5432/coffeedb
      SPRING_DATASOURCE_USERNAME: coffeeuser
      SPRING_DATASOURCE_PASSWORD: secretpassword
    depends_on:
      postgres-db:
        condition: service_healthy

volumes:
  postgres-data:
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

`depends_on` tek başına sadece veritabanı konteynerinin başladığını doğrular; veritabanı motorunun bağlantı kabul etmeye hazır olduğunu garanti etmez. Mutlaka `condition: service_healthy` ile birlikte kullanılmalıdır!

🇬🇧 WATCH OUT

A bare `depends_on` only waits for the container process to spawn, NOT for PostgreSQL to be ready! Always pair it with healthcheck conditions.

🎯 Özet Çıkarımlar

• Tek bir komut (`docker compose up -d`) ile tüm mikroservis yığını ayağa kalkar.

• Healthcheck blokları servisler arası yarış durumlarını (race condition) önler.

• Yerel geliştirme ortamını üretim ortamına en yakın hale getirir.

🎯 Key Takeaways

• Spawn entire multi-tier architectures with a single `docker compose up -d` command.

• Healthchecks eliminate startup race conditions between apps and databases.

• Mirrors production topology on local developer machines seamlessly.

6

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Container Güvenliği & İmaj Optimizasyonu: Distroless, Alpine & Non-Root

Container Security & Image Optimization: Distroless, Alpine & Non-Root

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Uçağa binerken yanınıza 150 kiloluk gardırop almazsınız; hem fazla para ödersiniz hem de hırsızlara davetiye çıkarırsınız! Standart Ubuntu imajında hacker'ların kullanabileceği 500 tane gereksiz araç (curl, apt, bash) vardır. Zırhlı mini imajda (Alpine veya Distroless) ise sadece Java çalışır. Üstüne bir de kapıya 'ROOT GİREMEZ' tabelası asarız!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't travel with a 150kg solid oak wardrobe on an airplane! Bloated images carry 500 unused utilities (bash, curl, package managers) that attackers love to exploit. Hardened minimal images (Alpine, Distroless) contain ONLY your Java binary. And we lock out the superuser by hanging a strict 'NO ROOT ALLOWED' sign!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Varsayılan olarak Docker konteynerleri `root` (UID 0) kullanıcısıyla çalışır. Konteynerden kaçış (container breakout) açığı oluşursa, saldırgan ana sunucuda da tam yetkili root olur! Bu yüzden: 1) Dockerfile içinde özel bir `appuser` (UID 10001) tanımlanmalıdır. 2) Paket yöneticisi ve kabuk (shell) bile içermeyen Google Distroless veya hafif Alpine imajları tercih edilmelidir. 3) Trivy veya Grype ile imajdaki CVE güvenlik açıkları CI pipeline'ında taranmalıdır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

By default, containers execute as `root` (UID 0). In the event of a Linux kernel or container breakout vulnerability, an attacker gains root on the physical host. Mitigation requires: 1) Enforcing non-root users (`USER 10001`), 2) Using shell-less Distroless bases, 3) Automated scanning for known CVEs via Trivy.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Güvenli Dockerfile Snippet:
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# 1. Root olmayan grup ve kullanıcı yarat
RUN addgroup -S appgroup -g 10001 && \
    adduser -S appuser -u 10001 -G appgroup

# 2. Sahipliği ver ve kullanıcıya geç
COPY --chown=appuser:appgroup target/app.jar app.jar
USER 10001:10001

# 3. Bellek sınırını JVM'e bildir (K8s pod çökmesini engeller)
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0 -XX:+UseG1GC"
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Konteyneri root olmayan kullanıcıyla çalıştırırken port numarasını 1024'ün altında (örneğin port 80) seçmeyin; Linux geleneksel olarak 1024 altındaki portları sadece root'a tahsis eder. Her zaman 8080 gibi yüksek portlar seçin!

🇬🇧 WATCH OUT

Non-root users cannot bind to privileged ports below 1024 (e.g. port 80). Always configure Spring Boot on high ports like 8080.

🎯 Özet Çıkarımlar

- Üretim ortamında ASLA konteynerleri root kullanıcısıyla çalıştırmayın.
- Distroless ve Alpine taban imajları saldırı yüzeyini ve dosya boyutunu küçültür.
- JVM `-XX:MaxRAMPercentage` ayarı bellek taşmalarını (OOMKilled) önler.

🎯 Key Takeaways

- NEVER execute containers as root in enterprise environments.
- Distroless and Alpine minimize the attack surface and download size.
- JVM container flags align Java heap limits with container cgroup limits.

BÖLÜM 2 / MODULE 2

Kubernetes ile Tanışma – Orkestra Şefi

Introduction to Kubernetes – The Conductor

Neden sadece Docker yetmez? K8s mimarisi, Control Plane, Worker Node'lar, Pod kavramı ve ilk Spring Boot manifestomuzu çalıştırma rehberi.

Why Docker alone falls short: K8s architecture, Control Plane, Worker Nodes, Pods, declarative models, and running your first Spring Boot manifest.

7

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

Neden Sadece Docker Yetmez? Kubernetes Neyi Çözer?

Why Docker Alone Isn't Enough: What Kubernetes Actually Solves

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Elinizde tek bir usta ve 1 fırın varken Docker harikadır. Ama ya restoranınız bir anda 10.000 şubeli bir dünya zincirine dönüşürse? Gece saat 03:00'te bir sunucunun fişi çekildiğinde, içindeki 50 konteyneri kim diğer sunuculara taşıyacak? Trafik 10 katına çıktığında kim anında 40 yeni konteyner açacak? İşte Kubernetes, 1.000 gemiyi ve 100.000 konteyneri tek bir parmak şıklatmasıyla yöneten baş amiraldir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Docker is great when managing a single coffee stall. But what happens when you run a global chain of 10,000 shops? If a server catches fire at 3 AM, who wakes up to resurrect the 50 crashed containers on another server? Who adds 40 extra containers when traffic surges 1000%? Kubernetes is the supreme admiral orchestrating a fleet of thousands of ships automatically!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker tek bir host (tekil sunucu) üzerinde konteyner çalıştırır. Kubernetes (K8s) ise onlarca hatta binlerce fiziksel sunucuyu tek bir devasa soyut süper-bilgisayar gibi birleştirir. Temel yetenekleri: 1) Automated Scheduling (Konteyneri en uygun kaynaklı sunucuya yerleştirme), 2) Self-Healing (Çöken konteynerleri yeniden başlatma, yanıt vermeyen düğümleri tahliye etme), 3) Horizontal Auto-Scaling (CPU/Trafiğe göre otomatik çoğalma), 4) Service Discovery & Load Balancing (Trafiği sağlıklı konteynerlere eşit dağıtma).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Docker manages containers on a single host. Kubernetes abstracts an entire fleet of physical/virtual machines into a single unified elastic compute fabric. Its core primitives deliver: 1) Automated scheduling & bin packing, 2) Self-healing & node drain automation, 3) Horizontal Pod Autoscaling (HPA), and 4) Automated discovery and Layer 4/7 load balancing.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Docker ile manuel çile (Sunucu çökerse bittiniz!):
ssh server-01 "docker run -d -p 8080:8080 my-coffee-app:1.0"

# Kubernetes ile bildirici güç (Sunucu patlasa da K8s başka sunucuda anında açar):
kubectl apply -f deployment.yaml
```

⚠️ Acemi Tuzağı & Pro İpucu

🇹🇷 AMAN DİKKAT

Kubernetes bir sihirli değnek değildir! Kötü yazılmış, bellek sızdıran veya 'graceful shutdown' desteklemeyen bir Spring Boot uygulamasını K8s'e koyarsanız sadece hatalarınızı daha hızlı çoğaltmış olursunuz.

⚠️ Gotchas & Pro-Tips

🇬🇧 WATCH OUT

Kubernetes does not fix broken code. If your Spring Boot app leaks memory or crashes on SIGTERM, K8s will just reboot and crash it faster.

🎯 Özet Çıkarımlar

- Docker konteynerleri çalıştırır; Kubernetes binlerce konteyneri orkestre eder.
- Kendi kendini iyileştirme (Self-Healing) sayesinde gece mühendis uykusundan uyanmaz.
- Yüzlerce sunucu tek bir dev işlemci ve bellek havuzuna dönüştürülür.

🎯 Key Takeaways

- Docker runs containers; Kubernetes orchestrates thousands across clusters.
- Self-healing keeps systems alive without manual midnight engineering interventions.
- Aggregates disparate physical nodes into a singular logical compute pool.

8

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

K8s Mimarisi: Control Plane (Beyin) vs Worker Nodes (Kas Gücü)

K8s Architecture: Control Plane (Brain) vs Worker Nodes (Muscle)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Dev bir inşaat şirketi düşünün. Şirketin lüks genel merkezinde (Control Plane) oturan yöneticiler vardır: 1) Resepsiyonist (API Server): Tüm emirleri kabul eder. 2) Noter (etcd): Yapılan her sözleşmeyi kasaya kilitler. 3) Teşrifatçı (Scheduler): Hangi işçinin hangi şantiyeye gideceğini seçer. 4) Denetmen (Controller Manager): Eksik işçi var mı diye kontrol eder. Şantiyede (Worker Node) ise baretini takıp harç karan işçiler (Kubelet ve Kube-Proxy) bulunur!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Picture a massive global construction firm. The air-conditioned headquarters is the Control Plane: 1) Receptionist (API Server): The only entry door for orders. 2) Master Vault (etcd): Immutable ledger storing all facts. 3) Dispatcher (Scheduler): Decides which worker goes to which site. 4) Inspector (Controller Manager): Constantly counts headcount. The actual construction sites are Worker Nodes with Kubelet (Site Foreman) and Kube-Proxy (Traffic Controller)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Control Plane bileşenleri: 1) kube-apiserver (REST API sunar; tüm bileşenler sadece onunla konuşur), 2) etcd (Raft konsensüs algoritması kullanan dağıtık anahtar-değer veritabanı; kümenin tek gerçek kaynağı), 3) kube-scheduler (Kaynak ihtiyaçlarına göre Pod'u en uygun Node'a atar), 4) kube-controller-manager (Deployment, Node, Endpoints kontrol döngülerini çalıştırır). Worker Node bileşenleri: 1) kubelet (Düğümdeki K8s ajanı; konteynerleri başlatır ve denetler), 2) kube-proxy (Ağ kurallarını ve iptables/IPVS yönlendirmelerini yönetir), 3) Container Runtime (containerd/CRI-O).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Control Plane internals: 1) kube-apiserver (central REST gateway; all cluster state passes here), 2) etcd (high-availability Raft-backed key-value store), 3) kube-scheduler (assigns unscheduled pods to optimal nodes), 4) kube-controller-manager (executes reconciliation loops). Worker Node components: 1) kubelet (node agent interacting with CRI to manage containers), 2) kube-proxy (maintains network packet filtering rules via iptables/IPVS), 3) Container Runtime (CRI-O / containerd).

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Control plane ve worker node'ların durumunu sorgula:
kubectl get nodes -o wide

# Cluster bileşenlerinin sağlık durumunu kontrol et:
kubectl get componentstatuses 2>/dev/null || kubectl get --raw='/readyz'
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

etcd veritabanı çökerse tüm küme kör olur! Üretim ortamında etcd her zaman tek sayılı (3 veya 5 düğüm) quorum mimarisiyle ve yüksek hızlı SSD disklerle çalıştırılmalıdır.

🇬🇧 WATCH OUT

If etcd loses quorum, your entire cluster enters read-only paralysis! Always run 3 or 5 odd-numbered etcd nodes on dedicated NVMe storage.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- API Server kümenin tek giriş kapısıdır; kimse doğrudan etcd ile konuşamaz.
- Kubelet, API Server'dan aldığı talimatları yerel konteyner motoruna iletir.
- Düğümler arızalansa bile Control Plane pod'ları başka düğümlere taşır.

- The API Server is the sole gateway; components never talk directly to etcd.
- Kubelet ensures containers described in PodSpecs are running and healthy.
- When a worker node dies, the Control Plane reschedules pods seamlessly.

9

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

En Temel Atom: Pod Nedir? Neden Doğrudan Konteyner Değil?

The Atomic Unit: What is a Pod and Why Not Bare Containers?

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

K8s dünyasında tek başına konteyner kiralayamazsınız; en küçük kiralık daire 'Pod'dur! Pod bir bezelye kabuğu, konteynerler ise içindeki bezelyelerdir. Çoğunlukla bir pod'da 1 bezelye (örneğin Spring Boot) yaşar. Ama bazen ana uygulamanın yanına ona yardımcı olan bir oda arkadaşı (Sidecar Konteyneri: örneğin log toplayıcı veya güvenlik ajanı) koyarsınız. Aynı pod'daki konteynerler aynı banyoyu (IP adresi) ve aynı buzdolabını (ortak disk) paylaşır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

In Kubernetes, you cannot rent a bare container; the smallest rentable apartment is a 'Pod'! A Pod is a pea pod, and containers are the peas inside. Usually, a pod hosts 1 main pea (your Spring Boot microservice). But sometimes, you pair it with a helpful roommate (Sidecar container: log shipper, envoy proxy). All containers in the same Pod share the exact same apartment IP and localhost loopback!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Pod, bir veya daha fazla konteynerin aynı Linux Network Namespace'ini, IPC namespace'ini ve depolama birimlerini paylaştığı soyut bir yapıdır. Bir pod ayağa kalktığında K8s önce 'pause' adında minik bir altyapı konteyneri başlatır. Bu pause konteyneri IP adresini ve ağ arayüzünü alır. Ardından gelen Spring Boot konteyneri ve sidecar konteynerleri bu ağ alanına katılır (`--net=container:pause`). Bu yüzden aynı pod içindeki iki farklı konteyner birbiriyle `localhost:port` üzerinden sıfır ağ gecikmesiyle konuşabilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A Pod encapsulates one or more co-located containers sharing Linux IPC, UTS, and Network namespaces via a lightweight 'pause' container. When a Pod initializes, the pause container claims the Pod IP. Subsequent app and sidecar containers attach to this namespace, allowing them to communicate over `localhost` with zero network serialization overhead and shared volumes.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Tek Pod içinde 2 konteyner örneği (Spring Boot + Log Tailer Sidecar):
apiVersion: v1
kind: Pod
metadata:
  name: coffee-with-sidecar
spec:
  containers:
    - name: spring-app
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      ports:
        - containerPort: 8080
    - name: log-shipper-sidecar
      image: alpine:latest
      command: ["sh", "-c", "while true; do echo 'Sidecar logging...'; sleep 10; done"]
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

İki bağımsız iş yapan mikroservisi asla aynı pod'un içine koymayın! Pod'lar birlikte ölür ve birlikte ölçeklenir. Sadece birbirinden ayrılamaz derecede kenetlenmiş yardımcı araçlar (proxy, log forwarder) sidecar yapılmalıdır.

🇬🇧 WATCH OUT

Never combine two independent microservices inside the same Pod! Containers in a pod scale together. Use sidecars strictly for tightly coupled helpers.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Pod, Kubernetes'in zamanladığı ve yönettiği en küçük konuşlandırma birimidir.
- Aynı Pod içindeki konteynerler localhost üzerinden haberleşir ve IP paylaşır.
- Çoğu senaryoda 1 Pod = 1 Uygulama Konteyneri kuralı geçerlidir.

- A Pod is the smallest atomic unit of scheduling in Kubernetes.
- Containers inside the same Pod share an IP address and reach each other on localhost.
- The standard pattern is 1 App Container per Pod, unless utilizing a Sidecar.

10

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

Bildirici (Declarative) Model vs Emirsel (Imperative) Model

Declarative Model vs Imperative Model: Desired State Philosophy

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Emirsel Model (Imperative): Garsona 'Mutfaktan bardağı al, 3 adım yürü, musluğu aç, suyu doldur, bana getir' demek gibidir. Bir adım aksarsa her şey kilitlenir. Bildirici Model (Declarative) ise şudur: 'Masanın üzerinde HER ZAMAN 3 bardak su istiyorum!' Biri bardağı kırarsa, garson hiç sormadan anında yeni bir bardak su koyar. K8s bildirici modelle çalışır: İstenen Durum (Desired State) = Gerçek Durum (Actual State)!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imperative Model: Shouting micromanaged commands: 'Walk 5 steps, grab glass, fill water, bring it'. If step 3 fails, the system crashes. Declarative Model: 'I want 3 glasses of water on this table AT ALL TIMES.' If an earthquake shatters a glass, the waiter immediately replaces it without being asked. That is Kubernetes: Desired State equals Actual State!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kubernetes'in kalbinde 'Reconciliation Loop' (Mutabakat Döngüsü) yatar. Siz bir YAML manifestosu verdiğinizde bu 'Desired State' (İstenen Durum) olarak etcd'ye yazılır. İlgili kontrolcü (örneğin DeploymentController) durmaksızın şu döngüyü çalıştırır: 1) Observe (Mevcut durumu oku: kaç pod çalışıyor?), 2) Diff (İstenen ile mevcut arasındaki farkı bul: 3 istenmiş, 2 var!), 3) Act (Farkı kapat: 1 yeni pod oluştur). Bu sayede dışarıdan bir pod manuel silinse bile K8s saniyeler içinde farkı onarır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes operates via an infinite 'Reconciliation Loop'. You declare the Desired State in YAML, stored in etcd. Controllers execute a continuous 3-step loop: 1) Observe actual cluster state, 2) Diff actual against desired state, 3) Actuate corrective API calls to converge actual state back to desired state. Self-healing is an inherent byproduct of this architecture.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Emirsel Komut (Tavsiye Edilmez - İz bırakmaz):
kubectl run my-coffee-pod --image=ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0

# Bildirici Manifest (Kurumsal Standart - Git'e konabilir):
# coffee-deployment.yaml dosyasını uygula:
kubectl apply -f coffee-deployment.yaml

# K8s istenen durum ile mevcut durum arasındaki farkı anında kapatır!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Canlı kümede doğrudan `kubectl edit` veya emirsel komutlarla değişiklik yaparsanız, bir sonraki GitOps veya `kubectl apply` çalışmasında bu manuel değişiklikler ezilir ve kaybolur!

🇬🇧 WATCH OUT

Making ad-hoc imperative edits directly in production leads to configuration drift; the next automated pipeline run will overwrite your changes.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Bildirici modelde 'nasıl yapılacağını' değil, 'neyin olması gerektiğini' söyleriz.
- Reconciliation döngüsü 7/24 istenen durum ile gerçek durumu eşitler.
- Tüm altyapı YAML dosyaları olarak Git'te versiyonlanabilir (Altyapı Kod Olarak - IaC).

- In declarative systems, you describe WHAT the state should be, not HOW to get there.
- The reconciliation loop runs 24/7, continuously eliminating drift.
- Declarative YAML manifests enable version-controlled Infrastructure as Code (IaC).

11

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

Yerel K8s Laboratuvarı: Minikube / Kind ve kubectl Temelleri

Local K8s Lab: Minikube / Kind and kubectl Command Mastery

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Koca bir Boeing 747 uçağını uçurmayı öğrenmek için hemen gerçek yolcu uçağına binmezsiniz; uçuş simülatöründe antrenman yaparsınız! Minikube veya Kind (Kubernetes IN Docker), laptopunuzun içinde güvenle çarpıp parçalayabileceğiniz minyatür bir Kubernetes kümesidir. `kubectl` ise bu uçağın direksiyonu ve vites koludur.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't learn to pilot a Boeing 747 on a live passenger flight; you practice in a flight simulator! Minikube and Kind (Kubernetes IN Docker) are zero-cost local simulators running right on your laptop where you can crash clusters safely. `kubectl` is the flight stick and control dashboard!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kind (Kubernetes-in-Docker), her bir K8s düğümünü (Node) ayrı bir Docker konteyneri olarak ayağa kaldırır. Çok düğümlü (multi-node) yerel kümeleri saniyeler içinde başlatmak için idealdir. `kubectl`, `~/.kube/config` dosyasındaki kimlik doğrulama sertifikalarını ve küme uç noktalarını kullanarak doğrudan `kube-apiserver` ile HTTPS üzerinden REST haberleşmesi yapar. Temel bağlamlar (Context) arasında hızlı geçiş sağlar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kind runs containerized Kubernetes nodes using Docker containers as mock hardware nodes, making multi-node cluster provisioning instantaneous. `kubectl` parses `~/.kube/config` to authenticate and route encrypted HTTPS REST requests directly to the cluster's `kube-apiserver` endpoint.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# 1. Kind ile yerel bir K8s kümesi oluştur
kind create cluster --name eli5-cluster

# 2. Küme durumunu ve düğümleri listele
kubectl cluster-info
kubectl get nodes

# 3. Kubeconfig bağlamını kontrol et
kubectl config current-context

# 4. Tüm isim alanlarındaki pod'ları listele
kubectl get pods -A
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Çok fazla yerel küme açıp kapatırsanız Docker diskiniz ve belleğiniz şişebilir. Kullanmadığınız Kind kümelerini `kind delete cluster --name eli5-cluster` ile silmeyi unutmayın.

🇬🇧 WATCH OUT

Running multiple local Kind clusters consumes heavy RAM. Always delete unused clusters via `kind delete cluster` when done.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Kind ve Minikube sıfır maliyetle profesyonel K8s geliştirme ortamı sağlar.
- `kubectl` komut satırı aracı API Server ile konuşan anahtardır.
- `~/.kube/config` birden fazla kümenin erişim anahtarlarını barındırır.

- Kind and Minikube provide rapid, zero-cost local sandbox environments.
- `kubectl` translates CLI commands into structured JSON REST requests to API Server.
- The kubeconfig file manages tokens and contexts across multiple clusters.

12

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

İlk Spring Boot Pod Manifestomuzu Yazma (YAML Anatomisi)

Writing Your First Spring Boot Pod Manifest (YAML Anatomy)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kubernetes'e bir dilekçe yazıyoruz! Bu resmi dilekçenin 4 kutsal bölümü vardır: 1) Hangi kanun maddesi? (`apiVersion`), 2) Ne istiyorsun? (`kind: Pod`), 3) Bu şeyin adı ne olsun? (`metadata: name`), 4) Özellikleri neler olsun? (`spec: containers`). İşte bu 4 parçayı doğru yazarsanız K8s dilekçenizi saniyeler içinde işleme koyar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Writing a Kubernetes YAML is filing an official petition to the cluster government! Every petition requires 4 sacred fields: 1) What law version? (`apiVersion`), 2) What resource type? (`kind: Pod`), 3) What is its name and tag? (`metadata`), 4) What are the exact specifications? (`spec`). Get those four right, and K8s grants your wish instantly!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Bir K8s manifestosunun yapısı: `apiVersion` (kaynağın API sürümü; örn: Pod için `v1`, Deployment için `apps/v1`), `kind` (oluşturulacak nesne tipi), `metadata` (name, namespace, labels, annotations gibi tanımlayıcı veriler), `spec` (kaynağın istenen teknik durumu; örn: imaj adı, portlar, ortam değişkenleri). `kubectl apply -f manifest.yaml` komutu verildiğinde API Server manifestoyu doğrular, etcd'ye kaydeder ve Kubelet konteyneri çeker.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The anatomy of every K8s resource: 1) `apiVersion` specifies the API endpoint group, 2) `kind` declares the schema type, 3) `metadata` houses names, namespaces, and queryable labels, 4) `spec` defines desired characteristics. Submitting via `kubectl apply -f` triggers schema validation in the API Server before etcd persistence.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# spring-boot-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: spring-coffee-pod
  labels:
    app: coffee-service
    env: dev
spec:
  containers:
    - name: coffee-container
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      ports:
        - containerPort: 8080
          name: http
      resources:
        requests:
          memory: "256Mi"
          cpu: "100m"

# Pod'u çalıştır ve loglarını izle:
# kubectl apply -f spring-boot-pod.yaml
# kubectl logs -f spring-coffee-pod
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

YAML dosyalarında TAB tuşu KESİNLİKLE yasaktır! Hizalama için sadece boşluk (space - genelde 2 boşluk) kullanılmalıdır, aksi takdirde YAML ayrıştırma hatası alırsınız.

🇬🇧 WATCH OUT

YAML syntax forbids TAB characters! Use strictly 2 spaces for indentation, or the API Server parser will reject the manifest with syntax errors.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Her K8s kaynağı apiVersion, kind, metadata ve spec alanlarını içerir.
- Labels (etiketler) nesneleri gruplamak ve filtrelemek için hayati önem taşır.
- Pod logları `kubectl logs -f <pod-name>` ile anlık olarak takip edilir.

- Every K8s object schema shares apiVersion, kind, metadata, and spec.
- Labels are key-value pairs enabling dynamic grouping, querying, and routing.
- Inspect application stdout/stderr streams directly using `kubectl logs`.

BÖLÜM 3 / MODULE 3

K8s İş Yükleri & Dayanıklılık

K8s Workloads & Self-Healing Resilience

Ölümsüz uygulamalar mimarisi: Deployments, ReplicaSets, kesintisiz güncellemeler, Spring Boot Actuator sağlık probları, StatefulSet ve kaynak kotaları.

Immortal applications architecture: Deployments, ReplicaSets, zero-downtime rolling updates, Spring Boot Actuator health probes, StatefulSets, and resource limits.

13

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

ReplicaSet ve Deployment: Ölümsüz Pod'lar

ReplicaSet and Deployment: Self-Healing Pods

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Tek başına bir Pod sokakta sahipsiz kalmış bir kedi yavrusu gibidir; arabaya çarparsa (çökerse) kimse arkasından ağlamaz! Deployment ise o kediye bakan 24 saat nöbetteki koruyucu melek ordusudur. 'Ben burada her zaman 3 adet sağlıklı Spring Boot kedisi istiyorum' dersiniz. Biri uyuyakalır veya ölürse, Deployment anında yenisini doğurur!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

A bare Pod is an unprotected orphan: if it crashes, nobody sheds a tear! A Deployment is its 24/7 guardian angel. You declare: 'I want exactly 3 healthy Spring Boot pods at all times.' If one pod gets killed or its server explodes, the Deployment controller instantly breathes life into an identical replacement!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Üretim ortamında neredeyse HİÇBİR ZAMAN çıplak Pod (bare pod) oluşturulmaz; her zaman Deployment kullanılır. Deployment hiyerarşisi şöyledir: Deployment -> ReplicaSet -> Pods. Deployment, yeni sürümler çıktığında yeni bir ReplicaSet açarak eski ReplicaSet'teki pod'ları kademeli olarak azaltır (Rolling Update). `spec.replicas` sayısı etcd'deki Desired State'tir. Pod çöktüğünde ReplicaSet controller bunu anında fark edip yeni bir Pod talebi oluşturur.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Bare pods are strictly anti-pattern in production; Deployments are mandatory. The architecture follows a strict hierarchy: Deployment -> ReplicaSet -> Pods. The Deployment controller manages declarative updates by spinning up a new ReplicaSet while scaling down the old one. If a container crashes, the Kubelet or ReplicaSet reconciles Pod counts back to the declared replica integer.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# spring-boot-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: coffee-deployment
  labels:
    app: coffee-app
spec:
  replicas: 3 # 3 adet yedekli pod çalıştır
  selector:
    matchLabels:
      app: coffee-app
  template:
    metadata:
      labels:
        app: coffee-app
    spec:
      containers:
        - name: coffee-container
          image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
          ports:
            - containerPort: 8080
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Deployment'ın `matchLabels` ile Pod şablonundaki (`template.metadata.labels`) etiketler birebir AYNI olmalıdır! Uyuşmazsa Deployment oluşturduğu pod'ları tanıyamaz ve sonsuz döngüde pod basmaya başlar.

🇬🇧 WATCH OUT

The `spec.selector.matchLabels` MUST match `spec.template.metadata.labels` identically. A mismatch causes the controller to lose track of its pods and trigger endless creation loops.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Deployment, pod'ların çökmesi durumunda anında kendi kendini iyileştirir.
- Arka planda ReplicaSet nesneleri sürüm geçmişini ve kopya sayısını yönetir.
- Tek bir komutla (`kubectl scale deployment ... --replicas=5`) anında ölçeklenir.

- Deployments guarantee self-healing: dead pods are instantly replaced.
- ReplicaSets handle the low-level replica counting and historical revisions.
- Scale your fleet on demand with `kubectl scale deployment --replicas=N`.

14

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

Sıfır Kesintiyle Güncelleme (Rolling Updates & Rollbacks)

Zero-Downtime Deployments: Rolling Updates & Rollbacks

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

200 km hızla giden bir yolcu treninde yolcuların çayı dökülmeden vagonları tek tek yenisiyle değiştirdiğinizi hayal edin! Rolling Update budur: 1 yeni vagon ekle, yolcuları oraya al, 1 eski vagonu emekli et. Yeni vagon bozuk çıkarsa tek bir düğmeye (`undo`) basarsınız ve saniyesinde eski vagonlara geri dönülür!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imagine replacing train cars on a locomotive speeding at 200 km/h without spilling a single passenger's hot tea! That is a Rolling Update: add 1 new modern carriage, move passengers, retire 1 old carriage. And if the new carriage has broken AC, press one panic button (`rollout undo`), and K8s reverts back instantly!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Deployment varsayılan strateji olarak `RollingUpdate` kullanır. İki kritik parametre vardır: 1) `maxSurge` (Güncelleme esnasında istenen kopya sayısının üzerine kaç ekstra pod açılabileceği; örn: %25 veya 1), 2) `maxUnavailable` (Güncelleme sırasında kaç pod'un geçici olarak devre dışı kalabileceği; 0 yapılırsa HİÇBİR ZAMAN servis kesintisi yaşanmaz). Hata durumunda `kubectl rollout undo deployment/coffee-deployment` ile anında önceki stabil ReplicaSet'e geri dönülür.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

RollingUpdate configuration governs zero downtime via two levers: 1) `maxSurge` (maximum pods created beyond desired replica target), 2) `maxUnavailable` (maximum pods that can be taken down during the roll). Setting `maxUnavailable: 0` ensures 100% capacity during deployments. Instant recovery is guaranteed via `kubectl rollout undo`.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Sıfır Kesinti Stratejisi Tanımı:
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0

# Sürüm Güncelleme:
kubectl set image deployment/coffee-deployment coffee-container=ghcr.io/eli5-k8s/spring-boot-k8s-eli5:2.0.0

# Durumu Canlı Takip Et:
kubectl rollout status deployment/coffee-deployment

# Hata Anında Tek Tuşla Geri Al (Rollback):
kubectl rollout undo deployment/coffee-deployment
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Eğer yeni sürümdeki Spring Boot uygulamanız veritabanında geriye dönük uyumsuz (breaking change) bir tablo şeması değişikliği yaparsa, K8s seviyesinde rollback yapsanız bile veritabanınız çökmüş kalır! Veritabanı şemaları her zaman geriye dönük uyumlu (expand and contract) olmalıdır.

🇬🇧 WATCH OUT

Kubernetes rollbacks cannot un-break a destructive database migration! Database schema changes must always maintain backward compatibility across rolling versions.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- `maxUnavailable: 0` ile sıfır kesintili (zero-downtime) sürümler garanti edilir.
- Tüm dağıtım adımları `kubectl rollout status` ile canlı izlenebilir.
- Tek bir komutla (`rollout undo`) saniyeler içinde eski sürüme dönülebilir.

- `maxUnavailable: 0` ensures capacity never dips below the replica threshold.
- Monitor rollout health live using `kubectl rollout status`.
- Instant rollbacks undo broken releases in seconds without redeploying code.

15

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

Spring Boot Sağlık Kontrolleri: Liveness, Readiness ve Startup Probes

Spring Boot Health Probes: Liveness, Readiness, and Startup

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Doktor muayenesi zamanı! K8s üç farklı muayene yapar: 1) Startup Probe: 'Bebek doğdu mu, nefes aldı mı? (Spring Boot JVM ısınana kadar sabret!)' 2) Liveness Probe: 'Hasta yaşıyor mu? Kalbi durduysa (kilitlendiyse) elektroşok ver ve pod'u yeniden başlat!' 3) Readiness Probe: 'Müşteri kabul edebilir mi? Veritabanı bağlantısı koptuysa, masaya müşteri göndermeyi geçici olarak durdur!'

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Time for the medical checkup! Kubernetes acts as a vigilant doctor with 3 stethoscopes: 1) Startup Probe: 'Has the baby woken up yet? (Hold your horses while the heavy Java JVM warms up!)' 2) Liveness Probe: 'Is the heart deadlocked? If flatlining, kill and reboot it!' 3) Readiness Probe: 'Can the kitchen take orders? If the database connection drops, temporarily stop routing diners here!'

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Spring Boot Actuator 2.3+ ile `/actuator/health/liveness` ve `/actuator/health/readiness` uç noktaları yerleşik olarak gelir. 1) Startup Probe: Ağır uygulamaların başlaması uzun sürerse liveness'ın pod'u erken öldürmesini engeller. 2) Liveness Probe: Thread deadlock veya bellek tıkanıklığında Kubelet pod'u yeniden başlatır (`restartCount++`). 3) Readiness Probe: Başarısız olursa pod ÖLDÜRÜLMEZ; sadece K8s Service Endpoints listesinden çıkarılır, böylece kullanıcılara 502/503 hatası gitmez.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Spring Boot Actuator natively exposes `/actuator/health/liveness` and `/actuator/health/readiness`. 1) Startup Probe protects slow-booting JVM apps from premature liveness kills. 2) Liveness probe failures instruct Kubelet to terminate and restart the container. 3) Readiness probe failures do NOT kill the container; instead, kube-proxy strips the Pod IP from the Service endpoint slice, shielding users from HTTP 500 errors.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# application.yml:
management:
  endpoint:
    health:
      probes:
        enabled: true

# deployment.yaml Probes Bloğu:
startupProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  failureThreshold: 30
  periodSeconds: 2 # 60 saniye boyunca açılışı bekle
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  periodSeconds: 5
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Liveness Probe içinde veritabanı veya dış servis kontrolü yapmayın! Veritabanınız çökerse, kümedeki 100 Spring Boot pod'unun tamamı aynı anda liveness hatası verip yeniden başlar ve tüm küme kilitlenir (Cascading Failure). Veritabanı kontrolü sadece Readiness probe'da olmalıdır!

🇬🇧 WATCH OUT

NEVER check external dependencies (database, Redis) in your Liveness Probe! If the database stumbles, all 100 app pods will simultaneously reboot in a catastrophic cascading failure.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Startup Probe JVM uygulamalarının açılışındaki erken ölümleri önler.
- Liveness kilitlenen uygulamayı yeniden başlatır (Restart).
- Readiness hazır olmayan pod'a trafiğin akmasını engeller (Zero 500 Errors).

- Startup probes give heavy JVM applications generous breathing room to initialize.
- Liveness restarts frozen or deadlocked application processes.
- Readiness isolates unready pods from live customer traffic safely.

16

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

StatefulSet: Veritabanları Neden Özel İlgi İster?

StatefulSets: Why Databases Demand Special Treatment

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Koyun sürüsü (Deployment) vs Safkan yarış atları (StatefulSet). Koyunların adı yoktur; hepsi birbirinin aynıdır, biri kaybolsa yerine rastgele bir koyun koyarsınız. Ama yarış atlarının adı (`postgres-0`, `postgres-1`), sabit bir ahırı (özel kalıcı diski) ve hiyerarşisi (1. at liderdir, diğerleri takipçidir) vardır! Veritabanları koyun değil, yarış atıdır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Cattle (Deployments) vs Thoroughbred Racehorses (StatefulSets). Cattle have no individual names; they are completely interchangeable. Racehorses have distinct names (`postgres-0`, `postgres-1`), assigned personal stables (dedicated permanent storage), and strict hierarchies (Leader vs Replica). Databases are pedigreed racehorses!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Deployment pod'ları rastgele isimler alır (`app-7d6f5-k9x2`) ve çöktüklerinde yeni bir rastgele isimle doğarlar. StatefulSet ise şunları garanti eder: 1) Öngörülebilir ve sabit isimler (`db-0`, `db-1`, `db-2`), 2) Sıralı başlatma ve kapatma (önce `db-0` hazır olur, sonra `db-1` başlar), 3) `volumeClaimTemplates` sayesinde her pod'a çöktüğünde bile peşinden gelen özel bir PersistentVolume tahsis edilir, 4) Headless Service ile her pod doğrudan kendi DNS adresiyle çağrılabilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

StatefulSets provide 4 crucial guarantees for stateful systems: 1) Deterministic, stable network identifiers (`stateful-0`, `stateful-1`), 2) Ordered, graceful deployment and scaling, 3) Dedicated persistent storage per replica via `volumeClaimTemplates`, 4) Headless Service integration providing stable per-pod DNS records for master-replica topology negotiation.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# statefulset-postgres.yaml (Özet)
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: "postgres-headless"
  replicas: 2
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
  volumeClaimTemplates:
    - metadata:
        name: db-storage
      spec:
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
            storage: 10Gi
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Bir StatefulSet'i sildiğinizde (`kubectl delete statefulset`), K8s veri kaybını önlemek için bağlı olan kalıcı diskleri (PVC) OTOMATİK OLARAK SİLMEZ! Diskleri elle silmeniz gerekir.

🇬🇧 WATCH OUT

Deleting a StatefulSet does NOT delete its associated PVC storage claims by design! Kubernetes protects database disks from accidental deletion.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- StatefulSet pod'ları sabit isim (`pod-0`, `pod-1`) ve sabit DNS alır.
- `volumeClaimTemplates` her pod kopyasına özel kalıcı disk bağlar.
- Veritabanı, Kafka ve Elasticsearch gibi durum tutan servisler için zorunludur.

- StatefulSet replicas possess persistent network hostnames and dedicated DNS.
- Each replica obtains its own dedicated PersistentVolume via templates.
- Essential for stateful clustered systems like PostgreSQL, Kafka, and Redis.

17

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

DaemonSet ve Job / CronJob: Nöbetçiler ve Gece Vardiyaları

DaemonSets, Jobs, and CronJobs: Watchmen and Night Shifts

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Üç farklı çalışan tipi: 1) DaemonSet: Her apartman bloğunun giriş kapısındaki sabit güvenlik görevlisidir (Her sunucuda istisnasız 1 adet çalışır; log ve virüs bekçisi). 2) Job: 'Git şu odayı boya ve işin bitince evine git' dediğiniz boyacıdır (Tek seferlik çalışır ve kapanır). 3) CronJob: Her gece saat 03:00'te gelip çöpleri toplayan gece temizlikçisidir.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Three specialized workers: 1) DaemonSet: The building security guard posted at every single entrance without exception (Runs exactly 1 pod per node for monitoring/logging). 2) Job: The painter hired to paint a wall and leave once finished (Run-to-completion batch). 3) CronJob: The night cleaner who arrives automatically every night at 3:00 AM to sweep the floors.

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

1) DaemonSet: Kümeye yeni bir Worker Node eklendiğinde K8s otomatik olarak o düğümde de 1 adet pod başlatır (FluentBit, Datadog ajanı, Calico node). 2) Job: Spring Boot veritabanı şema göçü (Flyway/Liquibase) gibi bir kez çalışıp başarıyla (exit code 0) tamamlanması gereken işleri yönetir. Başarısız olursa `backoffLimit` kadar tekrar dener. 3) CronJob: Linux crontab sözdizimiyle (`0 3 * * *`) periyodik Job'lar üretir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

1) DaemonSet guarantees that every eligible node runs a copy of a pod, auto-scaling as nodes join or leave (ideal for log collectors and CNI agents). 2) Job creates pods that run until a specified number of successful completions, managing retry backoffs for DB schema migrations. 3) CronJob schedules recurring batch jobs using standard 5-part crontab syntax.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# flyway-migration-job.yaml (Spring Boot DB Göçü)
apiVersion: batch/v1
kind: Job
metadata:
  name: coffee-db-migration
spec:
  backoffLimit: 3 # En fazla 3 kez tekrar dene
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - name: db-migrator
          image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
          command: ["java", "-jar", "app.jar", "--spring.flyway.migrate=true"]
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Job şablonunda `restartPolicy` varsayılan olarak 'Always' OLAMAZ! Bir Job ya 'OnFailure' ya da 'Never' olmalıdır, aksi takdirde biten iş sonsuza dek yeniden başlar.

🇬🇧 WATCH OUT

Job specs cannot use `restartPolicy: Always`! It must be `OnFailure` or `Never`, otherwise completed tasks loop infinitely.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- DaemonSet her sunucuda istisnasız 1 kopya çalıştırarak altyapıyı izler.
- Job'lar tek seferlik veritabanı göçleri ve veri işleme için idealdir.
- CronJob zamanlanmış periyodik rapor ve temizlik görevlerini otomatikleştirir.

- DaemonSets ensure uniform node coverage for infrastructure monitoring and logs.
- Jobs run tasks to completion with built-in retry policies.
- CronJobs reliably automate scheduled periodic batch workloads.

18

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

Kaynak Yönetimi: Requests ve Limits (Açgözlü Pod'ları Dizginleme)

Resource Management: Requests, Limits, and OOMKilled Taming

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Açık büfe restoran kuralı: 'Masanıza oturduğunuzda en az 1 dilim böreği garanti ediyoruz (Request). Ama tabağınıza 5 dilimden fazla koyarsanız (Limit), güvenlik gelir ve tabağınızı elinizden alıp sizi kapı dışarı atar (OOMKilled)!' Kaynak sınırları koymazsanız, obur bir Spring Boot pod'u tüm sunucunun RAM'ini yiyip diğer pod'ları açlıktan öldürür!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The Golden Buffet Rule: 'When seated, we guarantee you at least 1 steak (Request). But if you attempt to hoard more than 5 steaks (Limit), security grabs you by the collar and throws you into the street (OOMKilled)!' Without limits, a gluttonous Java pod will devour the entire server's RAM, starving innocent neighboring pods to death!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

K8s iki kaynak türünü yönetir: CPU (Sıkıştırılabilir kaynak: Limit aşılırsa süreç öldürülmez, CPU Throttling ile yavaşlatılır) ve Memory (Sıkıştırılamaz kaynak: Pod memory limitini 1 bayt bile aşarsa Linux OOM Killer devreye girer ve pod `Exit Code 137: OOMKilled` ile anında öldürülür). `requests` Scheduler'ın pod'u yerleştireceği düğümü seçerken baktığı garantili rezervasyondur. `limits` ise pod'un asla aşamayacağı tavandır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes differentiates compressible and incompressible resources: CPU is throttled via CFS bandwidth control when exceeding limits; Memory cannot be throttled—exceeding limits triggers an immediate Linux kernel SIGKILL (`Exit Code 137: OOMKilled`). `requests` dictate scheduling placement; `limits` set strict cgroup enforcement ceilings.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Pod Kaynak Tanımı:
resources:
  requests:
    cpu: "200m"     # 0.2 CPU Çekirdeği garanti et
    memory: "512Mi" # 512MB RAM garanti et
  limits:
    cpu: "1000m"    # En fazla 1 tam çekirdek kullanabilir (Aşarsa yavaşlar)
    memory: "1Gi"   # 1GB'ı aşarsa K8s ACIMASIZCA öldürür (OOMKilled)!

# JVM Bellek Ayarı (Limitin %75'ini Heap yap, gerisi Metaspace/Stack kalsın):
env:
  - name: JAVA_TOOL_OPTIONS
    value: "-XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0"
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

JVM varsayılan olarak sunucunun toplam fiziksel belleğini görebilir! Eğer `-XX:MaxRAMPercentage` veya `-Xmx` vermezseniz, Java 32GB'lık sunucunun 8GB'ını almaya çalışır, K8s limiti olan 1GB'a çarpıp anında OOMKilled olur!

🇬🇧 WATCH OUT

Without container-aware flags (`-XX:MaxRAMPercentage=75.0`), the JVM defaults to host RAM, swiftly violating the cgroup limit and triggering instant OOMKilled panics.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Requests zamanlama (scheduling) garantisidir; Limits ise azami tavandır.
- CPU aşımı yavaşlamaya (throttling), Bellek aşımı ani ölüme (OOMKilled) yol açar.
- Java uygulamalarında JVM bellek limitleri ile K8s limitleri tam uyumlu olmalıdır.

- Requests guarantee scheduling capacity; Limits enforce hard runtime caps.
- CPU spikes trigger throttling; Memory spikes trigger immediate OOMKilled termination.
- Always calibrate JVM MaxRAMPercentage to prevent unexpected container eviction.

BÖLÜM 4 / MODULE 4

Ağ, Keşif & Dış Dünyaya Açılma

Networking, Discovery & Ingress Routing

Pod'lar arası dinamik IP kaosuna son: ClusterIP servisleri, NodePort, LoadBalancer, Ingress kontrolcüleri, CoreDNS ve ConfigMap/Secret yönetimi.

Taming ephemeral IP chaos: ClusterIP services, NodePort, LoadBalancers, Ingress traffic cops, CoreDNS resolution, and externalizing configs via ConfigMaps & Secrets.

19

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Uçucu IP Çilesine Son: Service (ClusterIP) ve Pod Keşfi

Solving Ephemeral IPs: Services (ClusterIP) & Internal Discovery

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Pod'lar kelebekler gibidir; ömürleri kısadır, sürekli ölür ve her yeniden doğduklarında farklı bir telefon numarası (IP adresi) alırlar. Müşterilerinize sürekli değişen cep numaraları veremezsiniz! 'Service' (ClusterIP), şirketin asla değişmeyen sabit santral numarasıdır (Örn: 444 0 444). Müşteri santrali arar, santral arkada o an hangi garson müsaitse telefonu ona aktarır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Pods are butterflies: ephemeral and fragile. Every time a pod dies and resurrects, it receives a completely new IP address. You can't distribute changing phone numbers to customers! A Service (ClusterIP) is your company's immutable 1-800 hotline. Callers dial the single hotline number, and the switchboard automatically forwards calls to whoever is on duty!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

ClusterIP, sadece küme içinden erişilebilen sanal bir IP (VIP) ve DNS adıdır. Fiziksel bir ağ arayüzü yoktur! Arka planda `kube-proxy`, Linux çekirdeğindeki iptables veya IPVS kurallarını güncelleyerek ClusterIP'ye gelen paketleri hedef pod'ların IP'lerine rastgele (Round-Robin) yönlendirir. Servis, pod'ları `selector.matchLabels` ile bulur ve bu pod'ların canlı IP'lerini 'EndpointSlice' nesnelerinde güncel tutar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A ClusterIP is a virtual IP (VIP) resolvable strictly within the cluster. It has no physical network card! Instead, `kube-proxy` configures iptables or IPVS packet filter rules in the Linux kernel to rewrite destination IPs to matching pod IPs via round-robin. The Service maintains an active inventory of backing Pod IPs through `EndpointSlice` controller watchers.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# coffee-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: coffee-service # Küme içi DNS adı bu olur!
spec:
  type: ClusterIP
  selector:
    app: coffee-app # Bu etikete sahip pod'lara yönlendir
  ports:
    - name: http
      port: 80         # Servisin dinlediği dış kapı
      targetPort: 8080 # Spring Boot konteynerinin portu
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

ClusterIP servisine küme dışından (laptopunuzun tarayıcısından) doğrudan erişemezsiniz! Sadece küme içindeki diğer pod'lar veya `kubectl port-forward` ile erişilebilir.

🇬🇧 WATCH OUT

ClusterIP addresses are strictly internal! You cannot curl or open them in your desktop browser without port-forwarding or external ingress.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Service, dinamik pod IP'lerinin önüne sabit bir sanal IP ve DNS koyar.
- Trafik arka planda sağlıklı pod'lar arasında otomatik dengelenir.
- EndpointSlice nesneleri hazır olan pod'ların IP listesini anlık tutar.

- Services establish an immutable virtual IP and DNS fronting dynamic pods.
- Traffic load balances evenly across healthy pods matching label selectors.
- EndpointSlices continuously track ready and healthy pod endpoints.

20

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Dış Dünyaya İlk Kapı: NodePort ve LoadBalancer

Exposing Apps Outside: NodePort and Cloud LoadBalancers

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

ClusterIP ile evin içindesiniz. Peki sokaktaki müşteri içeri nasıl girecek? 1) NodePort: Binanın dış duvarına rastgele yüksek bir delik açıp kablo sarkıtmaktır (Port 30000-32767). 2) LoadBalancer: Binanın önüne profesyonel bir vale ve döner kapı koymaktır (AWS, Google Cloud veya Azure'dan gerçek bir genel IP kiralar)!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

ClusterIP keeps you locked indoors. How does a customer on the street enter? 1) NodePort: Drilling a hole in the outside wall with a designated high-numbered port (30000-32767). 2) LoadBalancer: Hiring an elite cloud-native valet company to manage your driveway with a shiny public static IP (AWS NLB, GCP Load Balancer)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

1) `NodePort`: Kümedeki TÜM işçi düğümlerin belirtilen portunu (örn: 31234) dinlemeye alır. `http://<HERHANGİ_BİR_DÜĞÜM_IP>:31234` adresine gelen istek servise aktarılır. 2) `LoadBalancer`: NodePort'un bir üst katmanıdır. K8s bulut sağlayıcısının (AWS/GCP/Azure) API'sini çağırarak otomatik bir bulut yük dengeleyici tahsis eder ve trafiği küme düğümlerine dağıtır. Yerel ortamlarda (Minikube/Kind) MetalLB kullanılabilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

1) `NodePort` opens a dedicated static port (30000-32767) on EVERY worker node. Packets hitting `<NodeIP>:<NodePort>` route into the service. 2) `LoadBalancer` extends NodePort by dispatching cloud provider API drivers (AWS ELB, GCP CLB) to provision a public IP routing external Internet traffic into the node ports.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# coffee-loadbalancer.yaml
apiVersion: v1
kind: Service
metadata:
  name: coffee-public-service
spec:
  type: LoadBalancer # Buluttan genel IP talep et!
  selector:
    app: coffee-app
  ports:
    - port: 80
      targetPort: 8080

# Minikube'da test etmek için tünel açma:
# minikube tunnel
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Her mikroservis için ayrı bir `LoadBalancer` servisi açarsanız, bulut sağlayıcınız her biri için ayda onlarca dolar fatura keser! 20 mikroservis için 20 ayrı IP almak yerine tek bir `Ingress` kullanılmalıdır.

🇬🇧 WATCH OUT

Provisioning a cloud LoadBalancer for every single microservice explodes cloud bills! Use a single Ingress Controller to route 50 services through 1 public IP.

🎯 Özet Çıkarımlar

- NodePort geliştirme ve test ortamlarında hızlı dış erişim sağlar.
- LoadBalancer gerçek dünya üretim ortamlarında genel IP sağlar.
- Trafik önce LoadBalancer'a, oradan NodePort'a, oradan ClusterIP'ye akar.

🎯 Key Takeaways

- NodePort is useful for quick debugging without cloud provider dependencies.
- LoadBalancers automate cloud-native public IP allocation and routing.
- Traffic hierarchically flows: LoadBalancer -> NodePort -> ClusterIP -> Pod.

21

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Ingress & Ingress Controller: Akıllı Trafik Polisi

Ingress & Ingress Controllers: The Smart Traffic Cop

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

50 katlı bir holding binasında her ofis için ayrı sokak kapısı yapamazsınız! Binanın tek bir ana döner kapısı ve lobisinde akıllı bir danışma görevlisi (Ingress Controller) olur. Gelen kişiye sorar: 'Kime geldiniz?' Kişi 'Kafeye (`/coffee`)' derse kafeye, 'Muhasebeye (`/billing`)' derse 5. kata yönlendirir. Üstelik kapıda kimlik kontrolü (SSL/TLS şifreleme) yapar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't build 50 separate front doors for a 50-story skyscraper! You have 1 grand lobby entrance with a sharp receptionist (Ingress Controller). Guests state their destination: 'Going to the coffee shop (`/coffee`)?' Turn left. 'Accounting (`/billing`)?' Elevator 4. Plus, the receptionist checks IDs and encrypts passes (HTTPS / SSL termination)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Ingress iki bileşenden oluşur: 1) Ingress Kaynağı (Kuralları belirten YAML dosyası), 2) Ingress Controller (Bu kuralları dinleyip uygulayan NGINX, Traefik, Envoy veya HAProxy pod'u). Ingress Controller, 7. katman (HTTP/HTTPS) yönlendirmesi yapar: Domain adına göre (`api.sirket.com` vs `web.sirket.com`) veya URL yoluna göre (`/api` vs `/auth`) trafiği ilgili ClusterIP servisine aktarır. SSL sertifikası tek merkezde sonlandırılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Ingress splits into: 1) Ingress Resource (declarative routing rules in YAML), 2) Ingress Controller (reverse proxy daemon like Ingress-NGINX or Traefik reconciling those rules). Operating at Layer 7 (HTTP/HTTPS), it terminates TLS, rewrites paths, and directs traffic based on SNI hostnames and URL path prefixes to backend ClusterIPs.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# coffee-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: coffee-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: coffee.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: coffee-service
                port:
                  number: 80
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Ingress YAML'ı yazmak tek başına HİÇBİR İŞE YARAMAZ! Kümenizde çalışan aktif bir Ingress Controller (örn: ingress-nginx) yoksa, yazdığınız Ingress kuralları sessizce havada asılı kalır.

🇬🇧 WATCH OUT

Applying an Ingress manifest without an active Ingress Controller running in your cluster does absolutely nothing; the resource remains dormant.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Ingress tek bir genel IP arkasında yüzlerce servisi akıllıca paylaştırır.
- Host-tabanlı ve Path-tabanlı 7. katman yönlendirmelerini destekler.
- SSL/TLS sertifikalarını tek merkezde yönetip güvenliği artırır.

- A single Ingress exposes hundreds of microservices behind 1 public IP.
- Supports host-based (virtual hosting) and path-based Layer 7 routing.
- Centralizes TLS termination and SSL certificate lifecycle management.

22

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Servisler Arası İletişim & CoreDNS (Spring Boot Entegrasyonu)

Inter-Service Communication & CoreDNS Resolution

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

K8s içindeki akıllı telefon rehberi: CoreDNS! Bir Spring Boot mikroservisi diğerine istek atarken IP adresini bilmek zorunda değildir. Rehbere 'Bana sipariş servisini bağla!' (`order-service`) der, CoreDNS anında doğru numarayı bağlar. Hatta başka bir departmandaysa tam adını söyler: `coffee-service.coffee-shop.svc.cluster.local`!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The internal yellow pages: CoreDNS! When your Spring Boot frontend calls your backend, it doesn't need to track IP addresses. It simply calls `http://coffee-service`! CoreDNS intercepts the DNS request and resolves the cluster IP instantly. Across namespaces, it uses the Fully Qualified Domain Name (FQDN): `service.namespace.svc.cluster.local`!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

K8s kümesindeki her Pod'un `/etc/resolv.conf` dosyasına CoreDNS servisinin IP'si otomatik yazılır. Bir pod `http://coffee-service` çağırdığında: 1) Yerel libc, search domain listesini (`coffee-shop.svc.cluster.local`, `svc.cluster.local`, `cluster.local`) sırayla dener. 2) CoreDNS eşleşen Servis VIP'sini döner. 3) Kube-proxy paketi hedef pod'a iletir. Spring Boot'ta `RestTemplate` veya `WebClient` ile doğrudan servis adı çağrılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Every Pod automatically inherits `/etc/resolv.conf` pointed at the cluster's CoreDNS service IP. Calling `http://coffee-service` triggers iterative DNS search domain appending (`<ns>.svc.cluster.local`). CoreDNS returns the Service VIP, and kube-proxy handles Layer 4 routing. In Spring Boot, configure your `WebClient` base URL directly with the service DNS name.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
// Spring Boot Java Servis Çağrısı Örneği:
@Service
public class OrderService {
    private final RestClient restClient;

    public OrderService(RestClient.Builder builder) {
        // CoreDNS adı doğrudan URL olarak kullanılır!
        this.restClient = builder
            .baseUrl("http://coffee-service.coffee-shop.svc.cluster.local")
            .build();
    }

    public CoffeeResponse fetchCoffee() {
        return restClient.get()
            .uri("/api/coffee")
            .retrieve()
            .body(CoffeeResponse.class);
    }
}
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Aynı namespace içindeki servisleri sadece adıyla (`http://coffee-service`) çağırabilirsiniz. Ama FARKLI bir namespace'teki servisi çağırmak için mutlaka namespace adını da eklemelisiniz (`http://coffee-service.diger-namespace`)!

🇬🇧 WATCH OUT

Calling a service in another namespace requires appending the namespace: `http://service-name.target-namespace`. Omitting it causes DNS 404 lookup failures.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- CoreDNS tüm servisler için otomatik DNS A/SRV kayıtları oluşturur.
- Tam alan adı: `<service-name>.<namespace>.svc.cluster.local` şeklindedir.
- Spring Boot kodunda IP adresi yazmak KESİNLİKLE yasaktır; DNS kullanılmalıdır.

- CoreDNS auto-provisions and maintains internal DNS records for all services.
- The internal FQDN syntax is `<service>.<namespace>.svc.cluster.local`.
- Never hardcode IP addresses in microservice configs; always utilize DNS names.

23

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Network Policies: K8s İçi Güvenlik Duvarı (Firewall)

Network Policies: Kubernetes Internal Micro-Firewalls

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Varsayılan olarak bir K8s kümesindeki her pod, diğer tüm pod'larla özgürce konuşabilir! Bu durum, oteldeki her müşterinin elini kolunu sallayarak mutfağa ve para kasasına girmesine benzer (Büyük tehlike!). NetworkPolicy, kapılara kilit takmaktır: 'Sadece beyaz şapkalı aşçılar (Backend) kasaya (Veritabanı) girebilir; müşteriler (Frontend) doğrudan giremez!'

💡 ELI5 Metaphor

🇬🇧 ENGLISH

By default, Kubernetes has a flat network: ANY pod can talk to ANY other pod! That's like letting hotel guests wander into the bank vault and restaurant freezer unhindered. A NetworkPolicy installs biometric security checkpoints: 'Only authenticated backend chefs can talk to the database; front-end visitors are completely blocked!'

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

NetworkPolicy, pod'ların ingress (gelen) ve egress (giden) ağ trafiğini etiketlere (labels), IP bloklarına (CIDR) veya namespace'lere göre filtreleyen kurallardır. DİKKAT: NetworkPolicy'nin çalışabilmesi için kümenizde Calico, Cilium veya WeaveNet gibi bir CNI (Container Network Interface) eklentisi kurulu olmalıdır. Varsayılan Flannel NetworkPolicy desteklemez. Kural yazıldığı anda hedef pod için 'varsayılan engelleme' (default-deny) aktif olur.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

NetworkPolicies enforce micro-segmentation at Layer 3/4 via Ingress/Egress packet filtering based on podSelector and namespaceSelector. CRITICAL: NetworkPolicies require an enforcing CNI plugin like Calico or Cilium. Without an enforcing CNI, policies are silently ignored! Declaring a policy toggles the selected pods into default-deny isolation.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# postgres-network-policy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-only-backend-to-db
  namespace: coffee-shop
spec:
  podSelector:
    matchLabels:
      app: postgres-db # Bu kural veritabanı pod'larını korur
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: coffee-app # SADECE coffee-app pod'ları gelebilir!
      ports:
        - protocol: TCP
          port: 5432
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Bir pod'a NetworkPolicy bağladığınız anda, açıkça izin vermediğiniz TÜM diğer gelen bağlantılar (diğer servisler, sağlık probları vb.) anında kesilir! Probeları veya DNS trafiğini (port 53) yanlışlıkla engellememeye dikkat edin.

🇬🇧 WATCH OUT

Applying a NetworkPolicy immediately locks down all unmentioned traffic! Forgetting to allow DNS egress (port 53 UDP/TCP) will break all network lookups.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Zero-Trust güvenliği için pod'lar arası erişim kısıtlanmalıdır.
- Calico veya Cilium gibi destekleyen bir CNI eklentisi zorunludur.
- Veritabanlarına sadece yetkili backend pod'larının erişmesi garanti edilir.

- Enforce Zero-Trust architecture by restricting lateral pod-to-pod movement.
- An enforcing CNI plugin (e.g. Calico, Cilium) is mandatory for enforcement.
- Guarantees databases only accept connections from verified backend services.

24

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

ConfigMap ve Secret: Konfigürasyonu Koddan Ayırma

Externalizing Config: ConfigMaps and Secrets with Spring Boot

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Telefon numaranız veya şifreniz değiştiğinde yeni bir telefon satın almazsınız! Aynı şekilde veritabanı şifresi değiştiğinde Spring Boot kodunu yeniden derleyip Docker imajı almamalısınız. ConfigMap: Herkesin görebileceği açık duyuru panosudur (API URL'leri, tema rengi). Secret: Kilitli çelik kasadır (Şifreler, kredi kartı API anahtarları)!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

When you change your Netflix password, you don't throw your TV in the trash and buy a new one! Similarly, when a DB password rotates, you don't recompile your Java code into a new Docker image. ConfigMaps are public bulletin boards (log levels, URLs). Secrets are locked steel safes (passwords, JWT signing keys)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Twelve-Factor App metodolojisinin 3. kuralı: 'Konfigürasyonu koddan ayır!' K8s iki nesne sunar: 1) ConfigMap: Düz metin anahtar-değer çiftleri, 2) Secret: Base64 kodlanmış hassas veriler. Bu veriler pod'a iki şekilde aktarılır: Ortam değişkeni (Environment Variable) olarak veya dosya olarak (Volume Mount). Spring Boot, `application.yml` içindeki `${DB_PASSWORD}` değişkenlerini bu sayede sıfır kod değişikliğiyle otomatik enjekte eder.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Twelve-Factor App principle III: Strict separation of config from code. 1) ConfigMaps store non-confidential key-value pairs. 2) Secrets store sensitive credentials base64-encoded (or encrypted at rest via KMS). Injected into pods either as environment variables (`envFrom`) or mounted as files. Spring Boot seamlessly binds them into `@Value` and `@ConfigurationProperties`.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# ConfigMap ve Secret'ı Pod'a Aktarma:
spec:
  containers:
    - name: coffee-app
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      env:
        - name: APP_MESSAGE # ConfigMap'ten tek değişken oku
          valueFrom:
            configMapKeyRef:
              name: spring-boot-config
              key: APP_MESSAGE
        - name: DB_PASSWORD # Secret'tan şifre oku
          valueFrom:
            secretKeyRef:
              name: spring-boot-secret
              key: DB_PASSWORD
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

K8s Secret'ları varsayılan olarak şifrelenmiş (encrypted) DEĞİLDİR; sadece Base64 ile encode edilmiştir! Yani `echo 'c2VjcmV0' | base64 -d` yazan herkes şifreyi görebilir. Gerçek güvenlik için etcd 'Encryption at Rest' veya HashiCorp Vault / AWS Secrets Manager kullanılmalıdır.

🇬🇧 WATCH OUT

Kubernetes Secrets are NOT encrypted by default; they are merely Base64-encoded! Anyone with read access can decode them. Enable KMS encryption-at-rest in etcd.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Aynı Docker imajı dev, test ve prod ortamlarında sıfır değişiklikle çalışır.
- Ortam değişkenleri veya dosya bağlama (volume mount) ile pod'a aktarılır.
- Secret'lar hassas verileri Git reposuna koyma felaketini önler.

- The same immutable container image runs across dev, stage, and prod via config injection.
- Configs can be mounted dynamically as filesystem directories or environment vars.
- Keeps passwords and private keys out of public Git version control repositories.

BÖLÜM 5 / MODULE 5

Depolama, Güvenlik & Otomatik Ölçekleme

Storage, Security & Horizontal Autoscaling

Kalıcı diskler (PV/PVC), StorageClass, RBAC erişim kontrolü, sıkı Pod güvenlik standartları, Namespace izolasyonu ve HPA ile otomatik ölçekleme.

Persistent storage (PV/PVC), StorageClasses, RBAC permissions, hardened Pod security contexts, Namespace tenancy, and HPA autoscaling under load.

25

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Kalıcı Depolama: PersistentVolume (PV) ve PersistentVolumeClaim (PVC)

Persistent Storage: PersistentVolumes (PV) and Claims (PVC)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Araba kiralama şirketi metaforu: 1) PersistentVolume (PV): Otoparkta bekleyen gerçek fiziksel arabadır (Sistem yöneticisinin hazırladığı 100GB'lık gerçek SSD disk). 2) PersistentVolumeClaim (PVC): Müşterinin elindeki kiralama kuponudur ('Bana 20GB'lık hızlı bir araba ver'). Kubernetes bu kuponu alır, otoparktaki uygun araba ile eşleştirir (Binding) ve anahtarı Spring Boot pod'una teslim eder!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The Car Rental Metaphor: 1) PersistentVolume (PV): The actual physical car parked in the garage (A real 100GB SSD provisioned by the infrastructure admin). 2) PersistentVolumeClaim (PVC): The customer's rental voucher ('I need a 20GB fast car'). Kubernetes matches the voucher to an available car (Binding) and hands the ignition keys to your Spring Boot Pod!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

PV küme çapında yönetilen gerçek depolama kaynağıdır (NFS, AWS EBS, Ceph). PVC ise bir pod'un talep ettiği depolama isteğidir (boyut ve erişim modu). Erişim modları: 1) `ReadWriteOnce` (RWO: Tek bir düğüm tarafından okunup yazılabilir; veritabanları için standarttır), 2) `ReadOnlyMany` (ROX: Birden fazla düğüm sadece okuyabilir), 3) `ReadWriteMany` (RWX: Birden fazla düğüm aynı anda yazabilir; NFS gerektirir). Pod PVC'yi bir volume olarak bağlar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

PV is a cluster-level resource backed by raw storage (AWS EBS, Ceph, NFS). PVC is a namespace-scoped ticket requesting capacity and access modes. Core modes: 1) `ReadWriteOnce` (RWO: mountable by a single node as read-write, ideal for databases), 2) `ReadOnlyMany` (ROX: concurrent read-only by multiple nodes), 3) `ReadWriteMany` (RWX: concurrent read-write, requires NFS/Gluster). K8s matches claims against volumes during the Binding lifecycle phase.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# coffee-pvc.yaml (Spring Boot Yüklenen Dosyalar İçin Kalıcı Disk Talebi)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: coffee-receipts-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi

# Pod İçinde Bağlama (Volume Mount):
# spec.containers[0].volumeMounts:
#   - mountPath: /app/uploads
#     name: receipts-storage
# spec.volumes:
#   - name: receipts-storage
#     persistentVolumeClaim:
#       claimName: coffee-receipts-pvc
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

`ReadWriteOnce` modu 'aynı anda sadece 1 pod yazabilir' DEĞİL, 'aynı anda sadece 1 sunucu düğümü (Node) yazabilir' anlamına gelir! Farklı sunuculardaki iki pod aynı anda RWO diski paylaşamaz.

🇬🇧 WATCH OUT

`ReadWriteOnce` means mountable by a single NODE, not a single Pod! Pods residing on different physical nodes cannot attach to the same RWO volume simultaneously.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Pod silinse veya başka düğüme taşınsa bile PVC'deki veriler korunur.
- Yazılımcılar PV detaylarını bilmeden sadece PVC ile disk talep eder.
- Veritabanları ve kalıcı dosya yüklemeleri için zorunludur.

- Data persists safely even if the hosting pod is evicted or rescheduled elsewhere.
- Decouples application developers (PVC) from storage administrators (PV).
- Essential for reliable stateful workloads, file uploads, and databases.

26

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

StorageClass ve Dinamik Disk Tahsisi

StorageClasses and Dynamic On-Demand Provisioning

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Eskiden bir yazılımcı disk istediğinde sistem yöneticisine e-posta atar, yönetici veri merkezine gidip kablo takardı (Statik Tahsis – 3 gün sürer!). StorageClass ise otomat makinesidir: Parayı atarsın (PVC talebi gönderirsin), makine sana saniyeler içinde AWS veya Google Cloud'dan gıcır gıcır bir SSD disk üretip verir (Dinamik Tahsis)!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

In the old days, when a developer needed a disk, they filed a ticket and waited 3 days for a sysadmin to rack a physical hard drive (Static Provisioning). A StorageClass is a vending machine: drop your token (PVC manifest), and the machine auto-provisions a shiny new SSD volume directly from the cloud provider in 5 seconds (Dynamic Provisioning)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

StorageClass, arkasındaki CSI (Container Storage Interface) sürücüsünü (örn: `ebs.csi.aws.com`) ve disk parametrelerini (IOPS, disk tipi: gp3, io2) tanımlar. Bir PVC `storageClassName: fast-ssd` belirttiğinde, Kubernetes ilgili bulut sağlayıcısının API'sini çağırarak diski dinamik olarak oluşturur, anında bir PV nesnesi yaratır ve PVC'ye bağlar. `volumeBindingMode: WaitForFirstConsumer` ayarı diskin pod'un zamanlandığı uygun bölgede (Availability Zone) açılmasını sağlar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

StorageClasses abstract CSI (Container Storage Interface) provisioners (e.g. AWS EBS CSI, GCP PD CSI). When a PVC specifies a storageClass, the CSI driver automatically creates cloud block volumes on-demand without manual PV creation. Setting `volumeBindingMode: WaitForFirstConsumer` delays volume provisioning until pod scheduling resolves, avoiding cross-zone attachment failures.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# storage-class-fast.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-nvme-ssd
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer # Pod hangi AZ'ye düşerse diski orada yarat!
parameters:
  type: gp3
  iops: "3000"
allowVolumeExpansion: true # İleride diski küçültmeden büyütebilme izni!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Farklı Kullanılabilirlik Alanları (Availability Zones): AWS EBS diskleri başka bir AZ'deki sunucuya bağlanamaz! `WaitForFirstConsumer` kullanmazsanız disk A bölgesinde açılabilir ama pod B bölgesine düşüp asılı kalabilir.

🇬🇧 WATCH OUT

Cloud block volumes are pinned to Availability Zones! Omitting `WaitForFirstConsumer` can provision a disk in us-east-1a while the pod is scheduled in 1b, causing permanent mount hangs.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Dinamik tahsis manuel disk oluşturma ve bağlama yükünü sıfıra indirir.
- `allowVolumeExpansion: true` ile diskler canlıda kesintisiz büyütülebilir.
- CSI sürücüleri K8s ile modern bulut depolama sistemleri arasındaki standart köprüdür.

- Dynamic provisioning eliminates manual disk creation and operational tickets.
- Enables online live volume expansion without deleting claims or pods.
- CSI plugins standardize integration across public clouds and on-prem storage.

27

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

K8s Güvenliği & RBAC: ServiceAccount, Role ve RoleBinding

Kubernetes Security: RBAC (Roles, Bindings & ServiceAccounts)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Holding binası güvenlik protokolü: 1) Personel Kimlik Kartı (ServiceAccount): Uygulamanızın yakasına taktığı kimlik kartıdır. 2) Yetki Kılavuzu (Role): 'Sadece 2. kattaki çay ocağına girebilir, kasaya dokunamaz' kuralları. 3) Yetki Ataması (RoleBinding): O kimlik kartını o kurallara zimmetleyen resmi tutanaktır. Bu üçlü olmadan hiç kimse API Server'a adım atamaz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Skyscraper Security Protocol: 1) Badge (ServiceAccount): The digital ID card worn by your Spring Boot application. 2) Rulebook (Role): 'Allowed to visit the cafeteria on Floor 2, strictly prohibited from opening the executive vault'. 3) Assignment (RoleBinding): The signed HR form linking the Badge to the Rulebook. Without RBAC, nobody touches the API Server!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kubernetes RBAC (Role-Based Access Control) API Server'a gelen istekleri denetler. Bileşenler: 1) `Subject` (Kullanıcı veya ServiceAccount), 2) `API Groups` (, `apps`, `batch`), 3) `Resources` (`pods`, `services`, `configmaps`), 4) `Verbs` (`get`, `list`, `watch`, `create`, `delete`). `Role` sadece belirli bir namespace içinde geçerlidir; `ClusterRole` ise küme genelinde (örn: Node'ları veya tüm namespace'leri okumak için) geçerlidir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes RBAC authenticates and authorizes API requests. Every rule consists of: API Groups, Resources, and HTTP-like Verbs (`get`, `list`, `watch`, `create`, `delete`). A `Role` binds permissions strictly inside a namespace; a `ClusterRole` spans the entire cluster (e.g. cluster-wide node monitoring). ServiceAccount tokens are mounted into pods at `/var/run/secrets/kubernetes.io/serviceaccount`.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# Spring Cloud Kubernetes için Salt-Okunur İzin:
apiVersion: v1
kind: ServiceAccount
metadata:
  name: spring-cloud-sa
  namespace: coffee-shop
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: configmap-reader
  namespace: coffee-shop
rules:
  - apiGroups: [""]
    resources: ["configmaps", "pods"]
    verbs: ["get", "list", "watch"] # Sadece okuma izni ver!
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bind-spring-reader
  namespace: coffee-shop
subjects:
  - kind: ServiceAccount
    name: spring-cloud-sa
roleRef:
  kind: Role
  name: configmap-reader
  apiGroup: rbac.authorization.k8s.io
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Uygulama pod'larına ASLA gereksiz yere `cluster-admin` yetkisi vermeyin! Spring Boot uygulamanızda bir güvenlik açığı (örn: Log4Shell) oluşursa, saldırgan tüm Kubernetes kümenizi ele geçirir (Least Privilege kuralı).

🇬🇧 WATCH OUT

NEVER grant `cluster-admin` to application pods! If an exploit strikes your Java code, the attacker inherits full administrative root over the entire cluster.

🎯 Özet Çıkarımlar

• En Az Ayrıcalık (Least Privilege) prensibiyle sadece gereken izinler verilmelidir.

• ServiceAccount uygulamaların K8s API'si ile güvenli konuşmasını sağlar.

• ClusterRole küme çapında, Role ise sadece tek bir namespace içinde geçerlidir.

🎯 Key Takeaways

• Always follow the Principle of Least Privilege: grant only essential verbs.

• ServiceAccounts provide secure identities for in-cluster API communication.

• ClusterRoles grant cluster-wide authority; Roles enforce namespace isolation.

28

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Pod Güvenlik Standartları: SecurityContext & Non-Root

Hardening Pods: SecurityContext & Non-Root Execution

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Konteyneri çelik bir zırh içine kilitlemek! Bir korsan Spring Boot uygulamanızın içine sızmayı başarsa bile elini kolunu bağlamak istiyoruz: 1) 'Root değilsin, garibansın!' (`runAsNonRoot`), 2) 'Sunucudaki hiçbir dosyayı değiştiremezsin!' (`readOnlyRootFilesystem`), 3) 'Hiçbir ayrıcalık talep edemezsin!' (`allowPrivilegeEscalation: false`). Korsan içeri girse bile hiçbir şey yapamaz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Locking the container inside an iron cage! Even if a hacker breaches your Spring Boot app, you want them handcuffed: 1) 'You are not root; you are a powerless user!' (`runAsNonRoot`), 2) 'You cannot alter any system files!' (`readOnlyRootFilesystem`), 3) 'You cannot elevate permissions!' (`allowPrivilegeEscalation: false`). The hacker is trapped and helpless!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Pod Security Standards (PSS) üç düzey tanımlar: Privileged, Baseline ve Restricted. Üretim ortamında 'Restricted' hedeflenmelidir. `securityContext` bloğunda: 1) `runAsUser: 10001` (Root olmayan UID), 2) `allowPrivilegeEscalation: false` (setuid ikili dosyalarıyla yetki yükseltmeyi engeller), 3) `capabilities: drop: ['ALL']` (Linux çekirdek yeteneklerinin tamamını kaldırır), 4) `readOnlyRootFilesystem: true` (Konteynerin kök dosya sistemini salt-okunur yapar; yazılması gereken `/tmp` klasörü `emptyDir` ile ayrıca bağlanır).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Pod Security Standards (PSS) define Privileged, Baseline, and Restricted profiles. Secure production pods must adhere to Restricted profiles: 1) `runAsNonRoot: true` with explicit UID (10001), 2) `allowPrivilegeEscalation: false`, 3) `capabilities.drop: ['ALL']` stripping Linux kernel capabilities, 4) `readOnlyRootFilesystem: true` with an ephemeral `emptyDir` mounted at `/tmp` for JVM temp writes.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Zırhlı Pod Güvenlik Bloğu:
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: coffee-app
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop:
            - ALL
      volumeMounts:
        - mountPath: /tmp
          name: tmp-volume # Tomcat geçici dosyaları için geçici RAM diski
  volumes:
    - name: tmp-volume
      emptyDir: {}
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Spring Boot (özellikle gömülü Tomcat) `/tmp` klasörüne dosya yazmaya ihtiyaç duyar. Eğer `readOnlyRootFilesystem: true` yapıp `/tmp` klasörüne bir `emptyDir` bağlamazsanız uygulamanız açılışta `IOException` ile patlar!

🇬🇧 WATCH OUT

Embedded Tomcat crashes if it cannot write PID files or sockets to `/tmp`. When enabling `readOnlyRootFilesystem: true`, always mount an `emptyDir` to `/tmp`.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Konteyneri root olmayan kullanıcıyla çalıştırmak container-breakout saldırılarını önler.
- `readOnlyRootFilesystem` zararlı yazılımların ikili dosya indirmesini engeller.
- Linux yeteneklerinin (capabilities) tamamı varsayılan olarak düşürülmelidir.

- Non-root execution prevents catastrophic container breakout host compromises.
- Read-only root filesystems block malware from persisting payloads to disk.
- Dropping ALL Linux capabilities minimizes OS kernel attack surfaces.

29

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Namespaces: Kümeyi Ortamlara Bölme (Dev, Staging, Prod)

Namespaces: Multi-Tenancy & Environment Isolation

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Dev bir alışveriş merkezini farklı mağazalara bölmek! Her mağaza (Namespace) kendi vitrinini düzenler. Bir stajyer 'Geliştirme' (Dev) mağazasındaki bir mankeni yanlışlıkla kırarsa, yan dükkandaki 'Üretim' (Production) kasasındaki paralar veya müşteriler bundan zerre kadar etkilenmez!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Partitioning a mega shopping mall into individual storefronts! Each store (Namespace) operates independently. If an intern accidentally knocks over a rack in the 'Dev' clothing shop, the real high-paying customers in the 'Production' jewelry store next door remain completely untouched and oblivious!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Namespace, fiziksel bir K8s kümesini mantıksal sanal kümelere böler. İsim çakışmalarını önler (farklı namespace'lerde aynı isimde `coffee-service` olabilir). İki kritik araçla desteklenir: 1) `ResourceQuota` (Namespace'in toplam CPU, RAM veya Pod sayısını sınırlar; örn: Dev namespace'i toplam en fazla 10 çekirdek CPU tüketebilir), 2) `LimitRange` (Pod başına varsayılan min/max kaynak sınırlarını zorunlu kılar).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Namespaces provide logical virtual partitioning within a single cluster. They prevent naming collisions and enforce governance via: 1) `ResourceQuota` (capping aggregate CPU, RAM, and object counts across the namespace), 2) `LimitRange` (enforcing default and min/max boundaries per container). Cluster-level resources (Nodes, StorageClasses, PVs) remain unnamespaced.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# dev-quota.yaml (Geliştirme Ortamı Kaynak Kotası)
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-compute-quota
  namespace: coffee-dev
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "10" # En fazla 10 pod açabilirler!

# Namespace oluştur ve aktif bağlamı değiştir:
# kubectl create namespace coffee-dev
# kubectl config set-context --current --namespace=coffee-dev
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Namespace'ler varsayılan olarak ağ düzeyinde yalıtım SAĞLAMAZ! Yani `coffee-dev` içindeki bir pod varsayılan ayarlarla `coffee-prod` içindeki bir pod'a ping atabilir. Tam izolasyon için mutlaka 23. Dersteki 'NetworkPolicy' kurallarını eklemelisiniz!

🇬🇧 WATCH OUT

Namespaces do NOT enforce network boundaries by default! Pods in `dev` can still ping pods in `prod` unless segmented with NetworkPolicies.

🎯 Özet Çıkarımlar

- Namespaces çok kiracılı (multi-tenant) ortamlarda düzen ve mantıksal sınır sağlar.
- ResourceQuota ekiplerin kümedeki tüm kaynakları sömürmesini engeller.
- `kubectl config set-context --current --namespace=...` ile varsayılan alan seçilir.

🎯 Key Takeaways

- Namespaces enable multi-tenant governance, organization, and name scoping.
- ResourceQuotas prevent rogue development teams from exhausting cluster capacity.
- Switch active context namespaces to avoid repetitive `-n <namespace>` flags.

30

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Yatay Otomatik Ölçekleme (HPA - Horizontal Pod Autoscaler)

Horizontal Pod Autoscaling (HPA): Traffic-Driven Fleet Scaling

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

E-ticaret sitenizde çılgın Cuma indirimi başladı! Sabah 3 pod ile başlayan Spring Boot servisiniz, binlerce müşteri akın edince terlemeye başladı (CPU kullanımı %80'e fırladı). HPA uykusundan uyanır, anında pod sayısını 3'ten 10'a çıkarır! Gece yarısı kampanya bitip müşteriler uyuyunca, fazla pod'ları sessizce kapatıp tekrar 3'e düşürür. Cebinizi boşuna bulut faturası ödemekten kurtarır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Black Friday flash sale madness! Your coffee delivery backend starts sweating as CPU utilization spikes past 80%. The Horizontal Pod Autoscaler (HPA) wakes up and automatically multiplies your pod count from 3 to 10 in 60 seconds! When midnight strikes and traffic dies, HPA scales the fleet back down to 3, saving you thousands on cloud bills!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

HPA, Metrics Server'dan aldığı metrikleri (CPU, Memory veya özel Prometheus metrikleri) sürekli sorgular. Ölçekleme formülü basittir: `İstenen Pod Sayısı = ceil(Mevcut Pod Sayısı * (Mevcut Metrik Değeri / Hedef Metrik Değeri))`. HPA ani dalgalanmalarda sistemin sürekli açılıp kapanmasını (flapping/thrashing) önlemek için 'cooldown/stabilization windows' (varsayılan: küçülme için 5 dakika bekleme) uygular.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

HPA queries metrics from the Metrics Server or Custom Metrics APIs. Its scaling algorithm calculates: `desiredReplicas = ceil[currentReplicas * (currentMetricValue / desiredMetricValue)]`. To prevent flapping (rapid oscillations), HPA enforces stabilization cooldown windows (defaulting to 5 minutes for downscaling) ensuring stable, predictable autoscaling behavior.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# coffee-hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: coffee-hpa
  namespace: coffee-shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: coffee-deployment
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70 # CPU %70'i geçerse yeni pod ekle!

# Yük testi yaparak HPA'yı canlı izle:
# kubectl get hpa -w
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

HPA'nın çalışabilmesi için Deployment manifestonuzda pod'lar için MUTLAKA `resources.requests.cpu` tanımlanmış olmalıdır! Request tanımlı değilse HPA CPU yüzdesini hesaplayamaz (`<unknown>/70%`) ve ölçekleme yapmaz.

🇬🇧 WATCH OUT

HPA requires `resources.requests` defined on container specs! Without requests, HPA cannot calculate percentage utilization and stays stuck in `<unknown>`.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- HPA trafik patlamalarında otomatik olarak yeni pod kopyaları açar.
- Sakin zamanlarda pod sayısını düşürerek bulut maliyetini minimize eder.
- Metrics Server kümede kurulu ve aktif olmalıdır.

- HPA automatically provisions extra replicas during unpredictable traffic spikes.
- Scales down automatically during idle periods to reduce operational infrastructure bills.
- Metrics Server must be deployed in the cluster to expose node and pod metrics.

BÖLÜM 6 / MODULE 6

k9s – Terminalin Matrix Arayüzü

k9s – The Terminal Matrix Cockpit

Terminalden ayrılmadan jet uçağı kokpiti gibi K8s yönetimi: k9s kısayolları, canlı log takibi, tek tuşla shell girişi, port yönlendirme ve özel eklentiler.

Supersonic terminal cluster navigation: k9s shortcuts, live log streaming, one-key shell access, port-forwarding, and custom productivity plugins.

31

BÖLÜM: K9S – TERMINALIN MATRIX ARAYÜZÜ | MODULE: K9S – THE TERMINAL MATRIX COCKPIT

k9s Nedir? Neden kubectl'den 10 Kat Daha Hızlıdır?

Introducing k9s: Why It's 10x Faster Than Raw kubectl

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Sürekli `kubectl get pods -n coffee-shop`, `kubectl describe pod ...`, `kubectl logs ...` yazmak, her kelime için daktiloda harf aramak gibidir. `k9s` ise terminalinizde açılan gerçek zamanlı, renkli bir Matrix kokpitidir! Tek bir tuşa basarak kümeyi izler, hatalı pod'ları kırmızı renkle anında yakalarsınız. Fareye gerek yok, parmaklarınız klavyede dans eder!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Typing `kubectl get pods -n my-ns`, `kubectl describe pod`, and `kubectl logs` 50 times an hour feels like writing code on an antique typewriter. `k9s` is a real-time, terminal-native supersonic fighter jet HUD! Red bars instantly scream at crashed pods; green glows mean peace. Zero mouse required–your fingers fly purely on muscle memory!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

k9s, Go diliyle yazılmış Terminal User Interface (TUI) tabanlı bir Kubernetes yönetim aracıdır. API Server ile kalıcı Watch Stream (gRPC/HTTP2) bağlantısı kurar. Her komutta API'ye yeni baştan sorgu atan `kubectl`'in aksine, k9s kümedeki olayları (events) anlık olarak yerel bellek önbelleğinde günceller. Bu da sorgu gecikmesini sıfıra indirir. `~/.kube/config` dosyanızı otomatik olarak okur.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

k9s is a high-performance terminal UI (TUI) written in Go. Rather than issuing stateless polling queries like `kubectl`, k9s establishes long-lived HTTP/2 Watch streams with the API Server, dynamically synchronizing state directly into its internal cache. It leverages your standard `~/.kube/config`, inheriting existing RBAC tokens and multi-cluster contexts seamlessly.

☕ Java Spring Boot Pratiği

```
# macOS Homebrew ile k9s kurulumu:
brew install derailed/k9s/k9s

# k9s'i başlatmak (Tek kelime yeterli!):
k9s

# Belirli bir namespace ile başlatmak:
k9s -n coffee-shop --context eli5-cluster
```

☕ Spring Boot in Action

⚠️ Acemi Tuzağı & Pro İpucu

🇹🇷 AMAN DİKKAT

k9s arka planda API Server ile sürekli canlı bağlantı (watch) kurduğu için, binlerce pod'un olduğu devasa kurumsal kümelerde çalıştırırken API Server üzerindeki CPU yükünü artırabilir. Çok büyük kümelerde `--all-namespaces` yerine spesifik namespace ile açmak önerilir.

⚠️ Gotchas & Pro-Tips

🇬🇧 WATCH OUT

In colossal enterprise clusters with tens of thousands of pods, watching all namespaces simultaneously can put strain on the API Server. Open k9s directly in a target namespace (`k9s -n target-ns`).

🎯 Özet Çıkarımlar

- k9s, CLI verimliliğini görsel bir arayüz hızında birleştiren terminal aracıdır.
- Gerçek zamanlı watch stream sayesinde değişiklikler anında ekranda parlar.
- Mevcut kubeconfig ve RBAC yetkilerini aynen miras alır.

🎯 Key Takeaways

- Combines the speed of the command line with the clarity of a visual dashboard.
- Displays real-time cluster state transitions instantly without polling.
- Inherits your active kubeconfig credentials, contexts, and RBAC restrictions automatically.

32

BÖLÜM: K9S – TERMINALIN MATRIX ARAYÜZÜ | MODULE: K9S – THE TERMINAL MATRIX COCKPIT

k9s Kısayolları ve Filtreleme: Işınlanma Sanatı

k9s Navigation & Filtering: The Art of Instant Teleportation

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

k9s'te gezinmek bir büyü gibidir! `:` tuşuna basıp `po` yazarsanız pod'lar listelenir, `deploy` yazarsanız deployment'lar açılır, `ns` yazarsanız tüm ortamlar önünüze dökülür. Arama yapmak için sadece `/` tuşuna basıp `coffee` yazmanız yeterlidir; aradığınız pod saniyesinde karşınızda!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Navigating k9s feels like instant teleportation! Press `:` and type `po` for pods, `deploy` for deployments, `svc` for services, `ns` for namespaces. To search, press `/`, type `coffee`, and press enter. You've filtered down 5,000 pods to your exact Spring Boot microservice in 0.5 seconds!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

k9s Vim benzeri tuş kombinasyonlarıyla (Keybindings) çalışır. Temel komutlar: `:` (Komut moduna geçer; K8s kısayol adlarını destekler: `:po`, `:svc`, `:ing`, `:cm`, `:sec`, `:cr`), `/` (Filtreleme modu; regex aramayı destekler), `ctrl-a` (Tüm namespace'leri tek ekranda gösterir), `0` (Sadece varsayılan namespace), `esc` (Bir önceki ekrana veya filtreyi temizlemeye yarar). Ok tuşları veya `j/k` ile listelerde gezinilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

k9s embraces Vim navigation paradigms. Primary bindings: `:` activates command mode (accepting shortnames like `:po`, `:svc`, `:deploy`, `:ing`, `:cm`), `/` activates regex-capable text filtering, `ctrl-a` aggregates all namespaces, `esc` cancels filters or navigates backward. Standard `j/k` or arrow keys move row cursors effortlessly.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# En Çok Kullanılan k9s Komut Modu Kısayolları:
:po        # Pod listesi
:deploy    # Deployment listesi
:svc       # Service listesi
:ing       # Ingress listesi
:ns        # Namespace seçici
:nodes     # Küme düğümleri ve kaynak durumları
:context   # K8s kümeleri arasında geçiş yap!
```

⚠️ Acemi Tuzağı & Pro İpucu

🇹🇷 AMAN DİKKAT

Bazen filtreleme yapıp (`/coffee`) aramayı açık unuttuğunuzda ekrandaki diğer pod'lar kaybolmuş gibi gelebilir. Panik yapmayın, `esc` tuşuna basarak filtreyi sıfırlayın!

⚠️ Gotchas & Pro-Tips

🇬🇧 WATCH OUT

If resources seem to have vanished, check if a filter is active! Press `esc` twice to clear the `/` search filter and reveal all workloads.

🎯 Özet Çıkarımlar

- `:` tuşu ile herhangi bir K8s kaynağına 1 saniyede ulaşılır.
- `/` tuşu ile canlı filtreleme yapılarak hedef pod saniyeler içinde bulunur.
- `ctrl-a` tüm namespace'leri tek pencerede toplayarak genel bakış sunar.

🎯 Key Takeaways

- The `:` key provides instant access to any standard or custom K8s resource type.
- The `/` key enables blazing fast real-time search across massive pod fleets.
- `ctrl-a` expands your viewport across all namespaces simultaneously.

33

BÖLÜM: K9S – TERMINALIN MATRIX ARAYÜZÜ | MODULE: K9S – THE TERMINAL MATRIX COCKPIT

Canlı Log İnceleme, Arama ve Pod İçine Giriş (`s` ve `l`)

Live Log Tailing, Searching & One-Key Shelling (`s` and `l`)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Gece saat 02:00'de üretimde bir hata çıktı! Spring Boot pod'unuzun loglarına bakmak için uzun uzun komut yazmazsınız. Pod'un üzerine gelin ve `l` tuşuna basın: Renkli loglar anında şelale gibi akar! Hata mı arıyorsunuz? `/Exception` yazın, bulsun. Pod'un içine girip dosyaları kontrol etmek mi istiyorsunuz? `s` tuşuna basın; anında pod'un içindeki terminaldesiniz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Midnight production outage! You don't fumble with lengthy CLI commands. Move the cursor over your Spring Boot pod and press `l`: colorized log streams cascade down your screen! Need to locate a bug? Press `/Exception` to jump straight to the stack trace. Need to verify environment configs? Press `s`: boom, you have an interactive shell inside the container!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

`l` (Logs) tuşu: K8s API'si üzerinden streaming log soketi açar. `0` tuşuyla logları baştan (head) sarabilir, `w` ile satır kaydırmayı (wrap) açıp kapatabilir, `t` ile zaman damgalarını (timestamps) ekleyebilirsiniz. `s` (Shell) tuşu: Pod içinde sırasıyla `bash`, `sh`, `/bin/sh` arar ve interaktif TTY oturumu başlatır (`kubectl exec -it`). Non-root çalışan konteynerlerde de sorunsuz çalışır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Pressing `l` (Logs) opens a streaming WebSocket log channel with colorized ANSI output, log wrapping toggle (`w`), and timestamp prefixes (`t`). Pressing `s` (Shell) launches an interactive TTY execution session into the container (`kubectl exec -it`), iteratively attempting `/bin/bash`, `/bin/sh`, and `/bin/ash` under the container's configured user identity.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# k9s Log ve Shell Kısayol Tuşları:
l         # Seçili pod'un canlı loglarını aç
p         # Önceki çöken pod'un loglarını gör (Previous Crashed Logs!)
s         # Pod içine terminal (Shell) aç
d         # Describe pod detaylarını aç (Event'leri görmek için!)
e         # Pod veya Deployment YAML'ını canlıda düzenle (Edit)
```

⚠️ Acemi Tuzağı & Pro İpucu

🇹🇷 AMAN DİKKAT

Eğer pod çökmüş ve `CrashLoopBackOff` durumundaysa, `l` tuşuna bastığınızda boş ekran görebilirsiniz. Çöken pod'un son nefesindeki hatayı görmek için mutlaka `p` (Previous Logs) tuşuna basmalısınız!

⚠️ Gotchas & Pro-Tips

🇬🇧 WATCH OUT

If a pod is caught in `CrashLoopBackOff`, normal logs might be empty! Press `p` to view PREVIOUS logs and read the fatal exception that killed the container.

🎯 Özet Çıkarımlar

- `l` tuşu pod loglarını anlık akış olarak ekrana getirir.
- `p` tuşu az önce çöken konteynerin hata günlüğünü (previous log) yakalar.
- `s` tuşu tek dokunuşla pod'un içine SSH benzeri güvenli shell bağlantısı kurar.

🎯 Key Takeaways

- `l` streams live application logs with zero configuration.
- `p` inspects the historical logs of terminated, crashed container instances.
- `s` instantly opens an interactive shell session into the running container.

34

BÖLÜM: K9S – TERMINALIN MATRIX ARAYÜZÜ | MODULE: K9S – THE TERMINAL MATRIX COCKPIT

Terminalden Port-Forwarding ve Pod Restart (`shift-f` ve `ctrl-d`)

Port-Forwarding and Workload Restarts (`shift-f` and `ctrl-d`)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

K8s içindeki kapalı bir Spring Boot servisini laptopunuzun tarayıcısında test etmek istiyorsunuz. Normalde karmaşık komutlar gerekir. k9s'te pod'un üzerine gelip `shift-f` tuşuna basarsınız; '8080:8080 tüneli açılsın mı?' diye sorar, enter'a basarsınız ve tarayıcınızda `localhost:8080` anında çalışır! Pod saçmaladı mı? `ctrl-d` tuşuna basın, anında öldürün; Deployment yenisini doğursun!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You want to test a strictly private internal Spring Boot pod on your laptop browser. Normally this takes annoying commands. In k9s, highlight the pod and press `shift-f`: it prompts 'Port 8080:8080?', press Enter, and your laptop is tunneled to the pod! Pod acting weird? Highlight it, press `ctrl-d` to kill it, and watch the Deployment instantly hatch a clean replacement!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

`shift-f` (Port-Forward): k9s yerel makinenizde bir TCP soketi açar ve K8s SPDY/HTTP2 port-forward API'si üzerinden hedef pod'un portuna tünel kurar. Aktif tünelleri görmek için `:pf` komutu yazılır. `ctrl-d` (Delete): Pod'u siler (`kubectl delete pod`). Eğer pod bir Deployment tarafından yönetiliyorsa, ReplicaSet saniyesinde yeni bir pod yaratır. Bir Deployment'ı kesintisiz yeniden başlatmak içinse Deployment ekranında `r` (Restart) tuşuna basılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Pressing `shift-f` invokes the SPDY/HTTP/2 port-forwarding subsystem, tunneling localhost sockets directly to internal pod ports without requiring NodePorts or Ingress. Type `:pf` to manage active forwards. Pressing `ctrl-d` deletes the highlighted resource. For Deployments, pressing `r` initiates a graceful `rollout restart`.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# k9s Tünel ve Yeniden Başlatma Kısayolları:
shift-f   # Port-forward tüneli aç
:pf       # Tüm açık tünelleri listele ve yönet
r         # Deployment üzerinde: Rolling Restart başlat!
ctrl-d    # Seçili Pod'u öldür (Kill / Delete)
```

⚠️ Acemi Tuzağı & Pro İpucu

🇹🇷 AMAN DİKKAT

Üretim ortamında (Production) yanlışlıkla bir Pod veya Deployment üzerindeyken `ctrl-d` tuşuna basmamak için k9s'in salt-okunur modunu (`k9s --readonly`) kullanabilirsiniz!

⚠️ Gotchas & Pro-Tips

🇬🇧 WATCH OUT

Prevent accidental deletions in production clusters by launching k9s in read-only mode: `k9s --readonly`.

🎯 Özet Çıkarımlar

- `shift-f` ile iç servisler saniyeler içinde localhost'a tünellenir.
- `:pf` menüsü tüm açık tünelleri görsel olarak izlemeyi ve kapatmayı sağlar.
- Deployment üzerindeyken `r` tuşuna basmak kesintisiz rolling restart tetikler.

🎯 Key Takeaways

- `shift-f` creates frictionless localhost port-forward tunnels in seconds.
- `:pf` provides centralized monitoring and teardown of active tunnels.
- Pressing `r` on a Deployment triggers an instant rolling restart.

35

BÖLÜM: K9S – TERMINALIN MATRIX ARAYÜZÜ | MODULE: K9S – THE TERMINAL MATRIX COCKPIT

Kaynak İzleme: Renkli CPU ve Bellek Göstergeleriyle Darboğaz Avı

Resource Monitoring: Visual CPU & Memory Bottleneck Hunting

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Sisteminiz neden yavaşladı? Hangi Spring Boot pod'u sunucunun tüm enerjisini emiyor? k9s pod listesinde CPU ve Bellek sütunlarını yeşil, sarı ve kırmızı renkli barlarla gösterir. Bir pod %95 belleğe yaklaştığında bar kıpkırmızı parlar: 'Beni kurtarın, birazdan OOMKilled olacağım!' diye bağırır. Dedektif gibi darboğazı yakalarsınız!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Why is the API suddenly sluggish? Which rogue Spring Boot pod is devouring server energy? k9s displays intuitive percentage bars next to CPU and Memory columns. When a pod reaches 95% RAM utilization, its gauge turns violently red, screaming: 'Help, I'm about to get OOMKilled!' You catch performance leaks like a master detective!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

k9s, Metrics Server API (`metrics.k8s.io`) ile entegre çalışır. Kaynak kullanımını iki farklı referansa göre gösterir: 1) `%REQUEST` (Pod'un rezerve ettiği taban kaynak kullanım yüzdesi), 2) `%LIMIT` (Pod'un tavan sınırına olan uzaklığı). Eğer bir Java pod'unun bellek barı sürekli artıyor ve hiç azalmıyorsa bu bir JVM Memory Leak (bellek sızıntısı) işaretidir. Node ekranında (`:nodes`) sunucu bazlı darboğazlar da anında izlenir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

k9s parses metrics directly from `metrics.k8s.io`. It dynamically evaluates utilization against both `%REQUEST` (guaranteed reservation) and `%LIMIT` (hard ceiling). If a Java pod's memory gauge climbs monotonically without garbage collection dips, you are observing a live JVM memory leak before OOMKilled evictions occur.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# k9s Ekranında CPU ve Bellek Göstergeleri:
# NAME            READY  STATUS   RESTARTS  CPU%  MEM%  CPU/REQ  MEM/REQ
# coffee-app-7b8c  1/1    Running  0         12%   84%   120m     430Mi
# (MEM% barı sarıya/kırmızıya dönüyorsa dikkat!)
```

⚠️ Acemi Tuzağı & Pro İpucu

🇹🇷 AMAN DİKKAT

Eğer CPU ve MEM sütunlarında `n/a` veya boşluk görüyorsanız, kümenizde `metrics-server` kurulu değildir! Minikube'da `minikube addons enable metrics-server` komutuyla aktif edebilirsiniz.

⚠️ Gotchas & Pro-Tips

🇬🇧 WATCH OUT

If CPU and MEM columns display `n/a`, your cluster lacks `metrics-server`! Enable it locally via `minikube addons enable metrics-server`.

🎯 Özet Çıkarımlar

- Renkli göstergeler sayesinde bellek darboğazları kriz çıkmadan fark edilir.
- JVM bellek sızıntıları (memory leaks) canlıda grafiksel olarak yakalanır.
- `:nodes` komutuyla sunucuların genel doluluk oranları denetlenir.

🎯 Key Takeaways

- Color-coded utilization bars spot resource exhaustion before crashes occur.
- JVM heap leaks are easily identified by observing continuous memory increases.
- The `:nodes` view shows host capacity and identifies overloaded hardware.

36

BÖLÜM: K9S – TERMINALIN MATRIX ARAYÜZÜ | MODULE: K9S – THE TERMINAL MATRIX COCKPIT

k9s Eklentileri ve Özel Kısayollar (Custom Plugins)

k9s Custom Plugins & Supercharged Productivity Hotkeys

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Uçağınızın kokpitine kendi özel kırmızı butonlarınızı eklemek! Örneğin tek bir tuşa (`ctrl-j`) basarak Spring Boot pod'unun içinden anında 'Java Thread Dump' almak veya log seviyesini tek tuşla `DEBUG` moduna geçirmek istiyorsunuz. k9s Plugins mekanizmasıyla istediğiniz özel komutu bir tuşa bağlayabilirsiniz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Adding custom red fire-buttons to your fighter jet cockpit! What if you could press `ctrl-j` on any pod to immediately dump JVM thread traces, or press `ctrl-b` to toggle Spring Boot log levels to `DEBUG`? k9s Plugins allow you to bind any custom shell or kubectl command to custom hotkeys!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

k9s eklentileri `~/.config/k9s/plugins.yaml` dosyasında tanımlanır. Bir eklenti şunları içerir: 1) `shortCut` (Tetikleyici tuş; örn: `shift-t`), 2) `scopes` (Hangi kaynakta geçerli; örn: `po`, `deploy`), 3) `command` ve `args` (Çalıştırılacak script). k9s çalışma anında `$NAMESPACE`, `$NAME`, `$POD`, `$CONTAINER` gibi ortam değişkenlerini komuta otomatik aktarır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

k9s plugins are defined declaratively in `~/.config/k9s/plugins.yaml`. Each plugin specifies a `shortCut`, target `scopes` (e.g. `pods`), and an executable `command` with arguments. k9s dynamically interpolates contextual variables like `$NAMESPACE`, `$NAME`, and `$CONTAINER` at runtime.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# ~/.config/k9s/plugins.yaml Örneği:
plugin:
  # Spring Boot Pod'undan Tek Tuşla Java Thread Dump Al!
  jvm-thread-dump:
    shortCut: Shift-T
    description: JVM Thread Dump
    scopes:
      - pods
    command: kubectl
    background: false
    confirm: false
    args:
      - exec
      - -it
      - -n
      - $NAMESPACE
      - $NAME
      - --
      - jcmd
      - "1"
      - Thread.print
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Eklentiler yerel bilgisayarınızdaki araçları (jcmd, curl, jq) çağırır. Eklentinin çalışabilmesi için ilgili aracın bilgisayarınızda veya konteyner içinde yüklü olması gerekir.

🇬🇧 WATCH OUT

Custom plugins invoke local or in-pod CLI binaries. Ensure required binaries (like `jcmd` or `jq`) exist before triggering custom hotkeys.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- k9s eklentileri günlük rutin bakım görevlerini tek tuşa indirir.
- Çalışma anı değişkenleri ($NAMESPACE, $NAME) dinamik komut yönetimini sağlar.
- Takım standartları için ortak `plugins.yaml` dosyaları paylaşılabilir.

- Custom plugins compress repetitive operational routines into single keypresses.
- Runtime context variables enable dynamic, context-aware command executions.
- Teams can standardize and share unified `plugins.yaml` across engineering orgs.

BÖLÜM 7 / MODULE 7

Red Hat OpenShift (OCP) – Kurumsal Kubernetes

Red Hat OpenShift (OCP) – Enterprise Kubernetes

Kurumsal bankaların ve dev telekomların tercihi: Standart K8s vs OpenShift, CoreOS, `oc` komutları, Routes, SCC güvenlik kısıtları ve S2I ile otomatik derleme.

The enterprise standard for banks and telcos: Vanilla K8s vs OpenShift, CoreOS, `oc` CLI, Routes, strict SCC security policies, and Source-to-Image (S2I) builds.

37

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

OpenShift Nedir? Standart K8s vs OpenShift

What is OpenShift? Vanilla Kubernetes vs Red Hat OpenShift

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Düz Kubernetes (Vanilla K8s), parçaları kutuda gelen bir yarış arabası şasisidir; motoru vardır ama koltukları, emniyet kemerini, klimayı, hatta kapı kollarını (Monitoring, Ingress, Registry, CI/CD) sanayiye gidip kendiniz tek tek taktırmak zorundasınız (Vahşi Batı!). Red Hat OpenShift ise fabrikadan zırhlı camları, deri koltukları, 24 saat yol yardımı ve tam güvenlik donanımıyla anahtar teslim çıkmış kurumsal bir Mercedes'tir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Vanilla Kubernetes is a DIY kit car chassis: you get an engine, but you must fabricate your own seats, seatbelts, AC, and mirrors (monitoring, ingress, registries, CI pipelines). Red Hat OpenShift is an armor-plated, fully assembled, factory-warrantied enterprise Mercedes delivered turnkey with 24/7 Red Hat enterprise support!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Red Hat OpenShift Container Platform (OCP), CNCF sertifikalı standart Kubernetes çekirdeği üzerine inşa edilmiş, tam teşekküllü kurumsal bir uygulama platformudur. Düz K8s'e ek olarak şunları yerleşik (out-of-the-box) getirir: 1) Yerleşik kurumsal Container Registry, 2) Prometheus/Grafana izleme yığını, 3) OpenShift Router (HAProxy tabanlı Route mekanizması), 4) Güçlü Web Konsolu (Developer ve Admin modları), 5) Otomasyonlu işletim sistemi (Red Hat Enterprise Linux CoreOS).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

OpenShift is a CNCF-certified Kubernetes distribution wrapped with integrated enterprise services. Beyond the core K8s API, OCP pre-packages: an integrated OCI image registry, full-stack Prometheus/Alertmanager monitoring, automated OS provisioning via RHEL CoreOS, HAProxy routing, Git-to-deployment pipelines (BuildConfigs), and a unified developer-centric Web UI.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Standart Kubernetes CLI:
kubectl get pods

# OpenShift CLI (Tam geriye dönük uyumlu + ek süper güçler!):
oc get pods
oc status
oc whoami
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Düz Kubernetes için yazılmış standart YAML dosyalarının %99'u OpenShift üzerinde doğrudan çalışır. Ancak OpenShift'in varsayılan güvenlik kısıtlamaları (SCC) nedeniyle `root` olarak çalışan konteynerler anında engellenir!

🇬🇧 WATCH OUT

While 99% of standard K8s manifests run on OpenShift, containers requiring root privileges will be immediately rejected by OpenShift's strict security policies.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- OpenShift, Kubernetes motoru üzerine inşa edilmiş anahtar teslim kurumsal platformdur.
- Geliştirici odaklı araçlar, izleme ve imaj depoları yerleşik olarak gelir.
- Dünyadaki büyük bankaların ve telekomünikasyon şirketlerinin 1 numaralı tercihidir.

- OpenShift is a fully supported, production-ready enterprise distribution of Kubernetes.
- Built-in integrated registry, monitoring, build pipelines, and routing out of the box.
- The enterprise standard across Fortune 500 banks, airlines, and telecom leaders.

38

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

OpenShift Mimarisi: CoreOS, OKD ve Platform Operatörleri

OpenShift Architecture: CoreOS, OKD, and Platform Operators

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Geleneksel sunucularda işletim sistemini güncellemek mayın tarlasında yürümek gibidir; bir paket uyuşmaz, tüm sunucu çöker. OpenShift ise Red Hat CoreOS adı verilen 'değiştirilemez' (immutable) bir işletim sistemi kullanır. Düğümlerde SSH ile dosya değiştiremezsiniz; tüm işletim sistemi bir konteyner imajı gibi tek parça halinde atomik olarak güncellenir! Kümenin her organını da özel bir robot (Cluster Operator) yönetir.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Upgrading standard Linux servers is like tiptoeing through a minefield; one conflicting library breaks everything. OpenShift runs on Red Hat Enterprise Linux CoreOS (RHCOS)—an immutable, container-optimized OS. You don't SSH into nodes to patch them; the entire OS updates atomically like an iPhone firmware update! Cluster Operators supervise every vital subsystem 24/7.

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

OpenShift 4 mimarisi iki temel kavrama dayanır: 1) RHCOS (Red Hat Enterprise Linux CoreOS): Salt-okunur kök dosya sistemi ve `ignition` ile provizyonlanan, Machine Config Operator (MCO) tarafından yönetilen işletim sistemi. 2) Cluster Operators: DNS, Network, Ingress, Authentication, Storage gibi tüm platform bileşenleri birer Kubernetes Operatörüdür. Cluster Version Operator (CVO), tüm kümenin sürüm yükseltmelerini tek bir bildirimle otomatik gerçekleştirir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

OpenShift 4 architecture centers on: 1) Immutable RHCOS provisioned via Ignition and managed entirely by the Machine Config Operator (MCO). 2) Self-driving Platform Operators: Network, DNS, Ingress, and Storage are all managed by specialized operators. The Cluster Version Operator (CVO) orchestrates end-to-end zero-downtime cluster upgrades declaratively.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Kümedeki tüm platform operatörlerinin sağlık durumunu incele:
oc get clusteroperators

# Küme sürüm durumunu ve yükseltme geçmişini sorgula:
oc get clusterversion
```

⚠️ Acemi Tuzağı & Pro İpucu

🇹🇷 AMAN DİKKAT

CoreOS düğümlerine girip manuel olarak `apt-get` veya `yum install` yapamazsınız! Kök dosya sistemi salt-okunurdur. Tüm sistem ayarları `MachineConfig` CRD'leri üzerinden yapılmalıdır.

⚠️ Gotchas & Pro-Tips

🇬🇧 WATCH OUT

Never attempt to manually patch RHCOS nodes via SSH. The root OS filesystem is read-only; all kernel tweaks must flow through `MachineConfig` manifests.

🎯 Özet Çıkarımlar

- RHCOS değiştirilemez işletim sistemi sayesinde düğümler asla 'drift' olmaz.
- Platform Operatörleri kümenin kendi kendini yönetmesini ve onarmasını sağlar.
- Tek tıkla tüm küme (işletim sistemi dahil) kesintisiz güncellenebilir.

🎯 Key Takeaways

- Immutable CoreOS prevents configuration drift across physical and virtual nodes.
- Platform operators manage cluster subsystems autonomously with self-healing loops.
- Enables seamless, one-click atomic updates spanning both K8s and the underlying OS.

39

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

`oc` Komut Satırı ve OpenShift Web Konsolu ile Görsel Yönetim

The `oc` CLI & Enterprise Web Console Experience

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

`oc` komut satırı aracı, `kubectl`'in siyah kuşak sahibi karate hocasıdır! `kubectl`'in tüm komutlarını anlar ama üstüne `oc new-app`, `oc login`, `oc project` gibi hayat kurtaran sihirli komutlar ekler. Web Konsolu ise bilim kurgu filmlerindeki kumanda merkezidir: Mikroservislerinizin durumunu birbiriyle konuşan renkli halkalar (Pod Donuts) halinde görsel olarak izlersiniz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The `oc` CLI is `kubectl` with a black belt in karate! It executes every single `kubectl` command natively, while introducing developer shortcuts like `oc new-app`, `oc status`, and `oc project`. Meanwhile, the OpenShift Web Console looks like a NASA mission control room: dynamic visual topology graphs display your Spring Boot pods as glowing, animated status rings!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

`oc` CLI ile OpenShift'e özgü kaynaklar (`routes`, `buildconfigs`, `imagestreams`, `deploymentconfigs`) yönetilir. `oc project coffee-shop` komutu, `kubectl config set-context` komutunun yaptığı karmaşık isim alanı geçişini tek hamlede yapar. Web Konsolu iki görünüme sahiptir: 1) Developer Perspective (Kod odaklı, Git bağlantılı, görsel topoloji haritası), 2) Administrator Perspective (Düğümler, depolama, RBAC ve küme operatörleri).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The `oc` binary provides first-class support for OpenShift custom resource definitions (`routes`, `imagestreams`, `buildconfigs`). Running `oc project <name>` simplifies namespace switching. The Web Console features two purpose-built modes: 1) Developer Perspective (visual topology maps, Git import wizards, log streams), 2) Administrator Perspective (cluster health, CRDs, quotas, hardware nodes).

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# 1. OpenShift kümesine giriş yap
oc login https://api.mycluster.openshift.com:6443 -u developer

# 2. Yeni bir proje (güçlendirilmiş namespace) oluştur ve geç
oc new-project coffee-shop

# 3. Projenin genel durumunu özetle
oc status

# 4. Pod ve servisleri listele
oc get all
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

OpenShift'te 'Project' aslında arka planda bir Kubernetes 'Namespace'idir. Ancak Project nesnesi ek olarak açıklama, görünen isim ve otomatik RBAC izinleri barındırır.

🇬🇧 WATCH OUT

In OpenShift, a 'Project' is a decorated Kubernetes Namespace with metadata annotations and pre-configured default RBAC roles.

🎯 Özet Çıkarımlar

- `oc` CLI `kubectl` ile %100 uyumludur ve ek kurumsal kolaylıklar sunar.
- Developer Web Konsolu mikroservis bağımlılıklarını görsel grafikle sunar.
- Tek bir komutla (`oc project`) ortamlar arasında zahmetsizce geçiş yapılır.

🎯 Key Takeaways

- The `oc` CLI is fully compatible with `kubectl` while adding high-productivity shortcuts.
- The visual topology console visualizes real-time microservice interactions intuitively.
- Switching environments is as simple as typing `oc project <name>`.

40

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

Routes vs Ingress: OpenShift Trafiği Nasıl Dışa Açar?

OpenShift Routes vs K8s Ingress: Edge TLS Demystified

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kubernetes dünyası henüz Ingress'i icat etmemişken, Red Hat mühendisleri 'Route' mekanizmasını geliştirmişti! K8s'te Ingress yazmak için önce gidip NGINX controller kurmanız, DNS kayıtlarıyla boğuşmanız gerekir. OpenShift'te ise tek bir komut verirsiniz (`oc expose svc coffee-service`), sistem size saniyesinde geçerli bir HTTPS adresi (`coffee-service-coffee-shop.apps.mycluster.com`) hediye eder!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Years before Kubernetes standardized Ingress, Red Hat engineered OpenShift Routes! In plain K8s, exposing an app requires installing an ingress controller, procuring domain names, and mapping DNS records. In OpenShift, you run ONE command (`oc expose svc coffee-service`), and OpenShift instantly creates a public, TLS-secured wildcard domain!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

OpenShift Route, küme girişindeki HAProxy tabanlı Router pod'ları tarafından yönetilir. Üç farklı TLS sonlandırma stratejisi sunar: 1) `Edge`: SSL sertifikası Route (HAProxy) seviyesinde sonlandırılır, arkadaki Spring Boot pod'una düz HTTP olarak iletilir (en yaygın ve hızlı model). 2) `Passthrough`: Şifrelenmiş SSL trafiği hiç açılmadan doğrudan pod'un içine kadar iletilir (uçtan uca şifreleme). 3) `Re-encrypt`: Route sertifikayı açar, inceler ve pod'a giderken farklı bir iç sertifikayla yeniden şifreler.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

OpenShift Routes are fulfilled by HAProxy routers. Three TLS termination strategies are supported: 1) `Edge`: TLS terminates at the HAProxy router, proxying plain HTTP to backend pods (highest performance), 2) `Passthrough`: Encrypted TLS packets pass through untouched directly to the pod's embedded SSL engine, 3) `Re-encrypt`: The router decrypts external traffic, inspects it, and re-encrypts using an internal certificate before reaching the pod.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# coffee-route.yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: coffee-route
  namespace: coffee-shop
spec:
  to:
    kind: Service
    name: coffee-service
  port:
    targetPort: http
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect # HTTP isteklerini otomatik HTTPS'e yönlendir!

# CLI ile tek komutta Route üretmek:
# oc expose service coffee-service --port=http
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

OpenShift aynı zamanda standart Kubernetes `Ingress` kaynaklarını da destekler (arka planda otomatik Route'a dönüştürür). Ancak kurumsal sertifika yönetimi için Route tanımlamak çok daha güçlü ve pratiktir.

🇬🇧 WATCH OUT

OpenShift automatically translates K8s Ingress objects into Routes under the hood, but native Routes provide finer-grained TLS and timeout controls.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Route, OpenShift'in yerleşik ve otomatik DNS destekli dışa açma mekanizmasıdır.
- `Edge`, `Passthrough` ve `Re-encrypt` ile esnek TLS mimarileri kurulur.
- HTTP trafiği `Redirect` politikasıyla otomatik olarak güvenli HTTPS'e yükseltilir.

- Routes provide out-of-the-box wildcard DNS and Layer 7 external routing.
- Flexible TLS termination models satisfy rigorous enterprise security compliance.
- Enforces automated HTTP-to-HTTPS redirects with a single line of YAML.

41

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

OpenShift Güvenliği: Security Context Constraints (SCC)

OpenShift Security: Security Context Constraints (SCC)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Neden Docker'da veya Minikube'da şakır şakır çalışan Spring Boot imajınız OpenShift'e atınca `Permission Denied` hatasıyla yere yapışır? Çünkü OpenShift aşırı titiz bir banka müdürü gibidir: 'Sen bana sormadan kendine root (UID 0) yetkisi vermişsin, hayatta izin vermem!' der. OpenShift her projeye rastgele bir kullanıcı kimliği (örneğin UID 1000620000) atar ve konteyneri o kimlikle çalışmaya zorlar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Why does a Docker image that runs perfectly on Minikube crash with `Permission Denied` on OpenShift? Because OpenShift enforces strict Security Context Constraints (SCC)! It says: 'You dare run as root (UID 0)? Forbidden!' OpenShift dynamically assigns a random high-numbered user ID (e.g. UID 1000620000) to every project and forces your container into it!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Security Context Constraints (SCC), Pod'ların kümede hangi ayrıcalıklarla çalışabileceğini belirleyen OpenShift güvenlik motorudur (K8s PodSecurity'nin atasıdır). Varsayılan SCC `restricted-v2`'dir: 1) Konteynerin `root` olarak çalışmasını engeller, 2) Rastgele tahsis edilen bir UID aralığı zorunlu kılar, 3) Host dizinlerini (`hostPath`) bağlamayı yasaklar. Çözüm: Dockerfile'da dosyaların grup sahipliğini `root (GID 0)` grubuna verip grup yazma izni tanımaktır (`chown -R 0 /app && chmod -R g=u /app`).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Security Context Constraints (SCC) control pod runtime permissions in OpenShift. The default `restricted-v2` SCC prohibits root execution, blocks hostPath mounts, drops capabilities, and dynamically allocates an arbitrary UID from the project's allocated UID range. Enterprise Dockerfiles comply by setting folder permissions to GID 0: `chown -R 0 /app && chmod -R g=u /app`.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# OpenShift Uyumlu Dockerfile Katmanı:
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/app.jar app.jar

# OpenShift rastgele bir UID ile çalıştırsa bile grubun (GID 0) yazma hakkı olsun:
RUN chown -R 0 /app && \
    chmod -R g=u /app

# Asla ROOT olarak çalıştırma!
USER 10001
ENTRYPOINT ["java", "-jar", "app.jar"]
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Acemi mühendisler hatayı çözmek için pod'a `oc adm policy add-scc-to-user anyuid -z default` vererek güvenlik duvarını delerler. Bu kurumsal denetimlerde (audit) affedilmeyen bir güvenlik ihlalidir! İmajınızı OpenShift standartlarına uygun hale getirin.

🇬🇧 WATCH OUT

Never grant `anyuid` SCC to bypass permission errors in production! Fix your container image file permissions to support arbitrary UIDs instead.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- SCC, konteynerlerin sunucu işletim sistemini ele geçirmesini kesin olarak engeller.
- OpenShift rastgele tahsis edilmiş dinamik UID'ler kullanarak güvenliği artırır.
- Uygulama klasörlerine GID 0 (`root group`) okuma/yazma izni verilmelidir.
- SCC prevents malicious containers from executing privileged host escalations.
- Dynamic random UIDs ensure isolation even if containers share hardware nodes.
- Supporting OpenShift requires granting GID 0 read/write access in your Dockerfile.

42

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

Source-to-Image (S2I) ile Spring Boot Kodunu Otomatik Dağıtma

Deploying Spring Boot via Source-to-Image (S2I) & Binary Builds

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Dockerfile yazmayı bile bilmeyen bir yazılımcı düşünün! OpenShift'e der ki: 'Al abi benim GitHub depomun linki burada, Java kodlarım içinde.' OpenShift sihirbaz gibi devreye girer: Kodu çeker, içine Maven ve JDK koyup derler, JAR'ı üretir, hafif bir runtime konteynerine koyar ve tek bir komutla canlıya alır! Buna S2I (Source-to-Image) denir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imagine a developer who has never written a Dockerfile in their life! They say to OpenShift: 'Here is my GitHub repository URL containing Java code.' OpenShift acts like a wizard: it clones the repo, provisions a Maven builder, compiles the JAR, bakes a lean container image, and deploys it in one breath! That is S2I (Source-to-Image)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

S2I mimarisi üç nesne ile çalışır: 1) `BuildConfig` (Derleme stratejisini, Git kaynağını ve tetikleyicileri tanımlar), 2) `ImageStream` (İmaj sürümlerini sanal olarak etiketleyen ve yeni imaj geldiğinde otomatik dağıtım tetikleyen soyutlama), 3) Builder Image (Red Hat'in sağladığı optimize `ubi8/openjdk-21`). Ayrıca lokaldeki `.jar` dosyasını doğrudan göndermek için 'Binary Build' (`oc start-build --from-file=target/app.jar`) yöntemi de sıklıkla kullanılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

S2I automates image creation via: 1) `BuildConfig` (defines Git source, webhooks, and builder images), 2) `ImageStream` (tracks container image metadata and triggers automated rollout upon updates), 3) Curated Red Hat OpenJDK builder images. Alternatively, teams use Binary Builds to push locally compiled JAR binaries directly into OpenShift build pods.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# 1. Java Git Deposundan Doğrudan Uygulama Üret (S2I):
oc new-app registry.access.redhat.com/ubi8/openjdk-21~https://github.com/my-org/spring-boot-k8s-eli5.git \
  --name=coffee-s2i-app

# 2. Derleme Loglarını Canlı Takip Et:
oc logs -f bc/coffee-s2i-app

# 3. Servisi Dış Dünyaya Aç:
oc expose svc/coffee-s2i-app
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Büyük kurumsal projelerde S2I küme üzerindeki CPU ve bellek kaynaklarını yoğun tüketebilir. Çok büyük ekiplerde derleme işlemini GitHub Actions gibi harici bir CI aracında yapıp OpenShift'e sadece hazır imajı iletmek daha verimlidir.

🇬🇧 WATCH OUT

Compiling multi-gigabyte Maven builds inside the cluster via S2I consumes cluster compute. For large teams, offloading compilation to GitHub Actions is best practice.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- S2I geliştiricilerin Dockerfile yazma zorunluluğunu ortadan kaldırır.
- ImageStreams yeni bir imaj etiketi geldiğinde otomatik dağıtımı tetikler.
- Kurumsal Red Hat UBI (Universal Base Image) güvenli temel sağlar.

- S2I abstracts container packaging away from application developers entirely.
- ImageStreams automatically trigger rolling deployments when new tags appear.
- Red Hat Universal Base Images (UBI) deliver enterprise-grade, certified runtimes.

BÖLÜM 8 / MODULE 8

Kubernetes Operators – Kendi Akıllı Robotunu Kodla

Kubernetes Operators – Build Your Own Robot SRE

Bir insan SRE mühendisinin beynini koda dökmek: CRD'ler, sonsuz mutabakat (reconciliation) döngüsü, popüler operatörler, OperatorHub ve Java Operator SDK (JOSDK).

Automating the human Site Reliability Engineer: Custom Resource Definitions (CRDs), the reconciliation loop, production operators, OperatorHub, and building Java operators.

43

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Operator Deseni Nedir? (Bir SRE Mühendisinin Beynini Koda Dökmek)

The Operator Pattern: Automating the Human SRE

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Şirketinizde çalışan kıdemli bir veritabanı uzmanını düşünün: Gece saat 03:00'te ana veritabanı çöktüğünde uyanır, yedek sunucuyu lider yapar, verileri eşitler, eski sunucuyu tamir edip havuza sokar. İnsan yorulur, uyur, hata yapar. Bir 'Operator' ise bu kıdemli mühendisin beynindeki tüm kuralları ve tecrübeyi 7/24 uykusuz çalışan bir yazılım robotuna dönüştürmektir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Picture a veteran database administrator in your company: When the primary database crashes at 3 AM, they wake up, promote the replica to master, synchronize data, and heal the cluster. Humans get sleepy and make typos. An 'Operator' is encapsulating that human engineer's operational wisdom into software code running 24/7 without ever taking a coffee break!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kubernetes standart olarak Deployment ve Service gibi temel durumsuz (stateless) iş yüklerini mükemmel yönetir. Ancak PostgreSQL, Kafka veya Redis gibi durum tutan (stateful) sistemler; sürüm yükseltme, otomatik yedekleme (backup), felaketten kurtarma (failover) ve şema göçü gibi alana özgü (domain-specific) karmaşık mantıklar gerektirir. Operator Pattern = Özel Kaynak Tanımı (CRD) + Özel Kontrolcü (Custom Controller).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes manages stateless workloads natively via Deployments. Stateful distributed systems (Postgres, Kafka, RabbitMQ) require domain-specific operational knowledge for backups, failover, and cluster topology negotiations. The Operator Pattern bridges this gap by combining: Custom Resource Definitions (CRD) to store desired state, and a Custom Controller executing reconciliation loops.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Operatörsüz manuel çile (50 adımda Postgres kümesi yönetimi):
# 1. SSH yap, 2. pg_dump al, 3. failover komutu ver...

# Operatör ile tek satırda otomatik küme ve yedekleme:
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: coffee-db-cluster
spec:
  instances: 3 # 3 düğümlü otomatik failover özellikli PostgreSQL!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Her şey için operatör yazmaya kalkmayın! Eğer uygulamanız standart bir REST API veya durumsuz mikroservis ise, standart bir K8s Deployment fazlasıyla yeterlidir. Operatörler karmaşık durum yönetimi (stateful) için tasarlanmıştır.

🇬🇧 WATCH OUT

Do not write operators for simple stateless apps! Standard Deployments are sufficient for 95% of microservices. Reserve operators for complex stateful automation.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Operator Deseni insan uzmanlığını yazılım otomasyonuna dönüştürür.
- Veritabanı yedeği, failover ve küme senkronizasyonunu otonom yönetir.
- CNCF ekosistemindeki en güçlü ve gelişmiş K8s genişletme mimarisidir.

- The Operator pattern translates human operational expertise into automated software.
- Automates complex stateful operations like failover, resizing, and backups.
- The premier architectural pattern for extending the native Kubernetes control plane.

44

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

CRD (Custom Resource Definition) ve Reconciliation Döngüsü

CRDs and the Infinite Reconciliation Loop

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kubernetes doğduğunda sadece 'Pod', 'Service' gibi birkaç kelime bilirdi. CRD (Custom Resource Definition), Kubernetes'e yepyeni kelimeler öğretmektir! 'Artık sen `SpringBootApp` veya `CoffeeMachine` kelimesini de bileceksin' deriz. Reconciliation (Mutabakat) Döngüsü ise robotun 'İstenen kahve hazırlandı mı? Eksik var mı?' diye etrafı sonsuz bir merakla kontrol etmesidir.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

When Kubernetes was born, it only understood basic words like 'Pod' and 'Service'. A CRD (Custom Resource Definition) teaches Kubernetes brand-new nouns! You say: 'From now on, you understand what a `SpringBootApp` is.' The Reconciliation Loop is the robot's infinite inner monologue: 'I see 1 desired `SpringBootApp`; do 3 matching pods exist? If not, make it so!'

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

1) CRD: K8s API Server'a OpenAPI v3 şemasıyla yeni bir REST uç noktası (`/apis/apps.eli5.io/v1alpha1/...`) ekler. 2) Custom Resource (CR): Bu şablon kullanılarak oluşturulan gerçek nesnedir. 3) Reconciliation Loop: Controller sürekli olarak K8s API'sindeki Watch kuyruğunu dinler. `Reconcile(Request)` fonksiyonu çağrıldığında: Mevcut kaynakları sorgular, aradaki farkı hesaplar ve K8s API çağrılarıyla sistemi olması gereken duruma eşitler.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

1) CRDs register custom REST endpoints into the API Server with OpenAPI v3 schema validation. 2) Custom Resources (CR) are instances conforming to that schema. 3) The Reconciliation Loop triggers on Watch event queues. The `Reconcile(Request)` method retrieves the observed state, calculates the delta against desired state, and issues idempotent API calls to eliminate drift.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# springbootapp-crd.yaml (Yeni Nesne Tanımı)
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: springbootapps.apps.eli5.io
spec:
  group: apps.eli5.io
  versions:
    - name: v1alpha1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                image: { type: string }
                replicas: { type: integer, minimum: 1, maximum: 10 }
                port: { type: integer, default: 8080 }
  scope: Namespaced
  names:
    plural: springbootapps
    singular: springbootapp
    kind: SpringBootApp
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Reconciliation döngüsündeki kodunuz MUTLAKA 'Idempotent' (Tekrarlanabilir) olmalıdır! Döngü 1 kez de çalışsa, peş peşe 100 kez de çalışsa aynı sonucu üretmeli, her seferinde sıfırdan yeni kaynak yaratmamalıdır.

🇬🇧 WATCH OUT

Reconciliation logic MUST be strictly idempotent! Whether executed once or a thousand times, it must evaluate current state and produce the exact same outcome.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- CRD'ler Kubernetes API'sini kendi alan adınıza özel nesnelerle genişletir.
- Reconciliation döngüsü 'İstenen Durum' ile 'Mevcut Durum'u daima eşitler.
- OpenAPI v3 doğrulaması hatalı YAML girişlerini anında reddeder.
- CRDs extend the core Kubernetes API with domain-tailored schema endpoints.
- The reconciliation loop relentlessly drives actual state toward desired state.
- Built-in OpenAPI v3 validation rejects malformed YAML payloads at the API door.

45

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Dünyayı Yöneten Hazır Operatörler: Prometheus, Postgres, Strimzi

Battle-Tested Operators: Prometheus, CloudNativePG, and Strimzi

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Tekerleği baştan icat etmeyin! Dünyanın en iyi mühendisleri zaten en zor işler için mükemmel robotlar yazdı: 1) Prometheus Operator: Kümedeki tüm pod'ları otomatik keşfedip grafiklerini çizer. 2) CloudNativePG: PostgreSQL veritabanını tek satırla kurar, saniyede bir yedek alır, ana sunucu çökerse 3 saniyede yedeği devreye sokar. 3) Strimzi: Gezegenin en karmaşık sistemi olan Apache Kafka'yı çocuk oyuncağına çevirir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Don't reinvent the wheel! The world's top infrastructure architects have already coded elite robots for the hardest problems: 1) Prometheus Operator: Dynamically sniffs out microservices and plots metrics dashboards. 2) CloudNativePG: Deploys HA PostgreSQL, takes automated point-in-time backups to S3, and fails over in 3 seconds flat. 3) Strimzi: Tames Apache Kafka clusters effortlessly!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Bu operatörler üretimde altın standarttır: 1) CloudNativePG (`postgresql.cnpg.io`): PostgreSQL streaming replication, PgBouncer bağlantı havuzu, WAL arşivleme ve S3/MinIO yedeklemelerini CRD'lerle tam otomatik yürütür. 2) Prometheus Operator (`monitoring.coreos.com`): `ServiceMonitor` CRD'si ile Spring Boot Actuator `/actuator/prometheus` uç noktasını otomatik olarak Prometheus hedeflerine ekler. 3) Strimzi (`kafka.strimzi.io`): Kafka broker ve Zookeeper/KRaft topolojisini yönetir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Production staples: 1) CloudNativePG manages PostgreSQL physical streaming replication, seamless zero-downtime rolling upgrades, and automated barman-based WAL archiving to S3. 2) Prometheus Operator utilizes `ServiceMonitor` CRDs to auto-discover Spring Boot `/actuator/prometheus` endpoints. 3) Strimzi manages Kafka brokers, topics, and users purely via declarative YAML.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# servicemonitor-spring.yaml (Prometheus için Spring Boot İzleme Kuralı)
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: spring-boot-monitor
  namespace: coffee-shop
spec:
  selector:
    matchLabels:
      app: coffee-app
  endpoints:
    - port: http
      path: /actuator/prometheus
      interval: 15s # Her 15 saniyede bir JVM metriklerini topla!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Operatör kurarken CRD'lerin küme geneli (Cluster-wide) yetki gerektirdiğini unutmayın! Standart bir namespace geliştiricisi tek başına CRD kuramaz; cluster-admin yetkisi gerekir.

🇬🇧 WATCH OUT

Installing operators requires cluster-admin privileges because CRD installation is a cluster-scoped operation affecting all tenants.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Popüler açık kaynak operatörler yüzlerce saatlik operasyonel emeği kurtarır.
- `ServiceMonitor` nesnesi Spring Boot metriklerini otomatik Prometheus'a bağlar.
- Veritabanı felaket senaryoları insan hatası olmadan otomatik çözülür.

- Battle-tested open-source operators save hundreds of hours of maintenance toil.
- ServiceMonitors auto-scrape Spring Boot Micrometer metrics without static configs.
- Automated database failover eliminates catastrophic human mistakes during outages.

46

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

OperatorHub ve Operator Lifecycle Manager (OLM)

OperatorHub & Operator Lifecycle Manager (OLM): The K8s App Store

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Akıllı telefonunuza nasıl App Store veya Google Play'den tek tıkla uygulama indiriyorsanız, Kubernetes kümenize de 'OperatorHub' üzerinden tek tıkla kurumsal operatörler yüklersiniz! OLM (Operator Lifecycle Manager) ise telefonunuzun arka plandaki otomatik güncelleme yöneticisidir: Operatörlerin yeni sürümleri çıktığında onları kesintisiz günceller!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Just like browsing the App Store or Google Play to install apps with a single tap, 'OperatorHub' is the official enterprise app marketplace for Kubernetes! OLM (Operator Lifecycle Manager) is the automated app manager running under the hood: when a database operator releases a security patch, OLM upgrades it smoothly without breaking a sweat!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

OLM mimarisi 4 temel kaynaktan oluşur: 1) `CatalogSource` (Mevcut operatörlerin ve sürümlerin listelendiği depo), 2) `Subscription` (Hangi operatöre abone olunduğunu ve güncelleme kanalını - channel: stable/fast - belirtir), 3) `InstallPlan` (Gerekli CRD'leri ve RBAC izinlerini hesaplayan yürütme planı), 4) `ClusterServiceVersion` (CSV: Operatörün tüm manifestolarını, konteyner imajlarını ve metaverilerini paketleyen ana nesne). OpenShift'te yerleşik gelir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

OLM manages operator lifecycles via: 1) `CatalogSource` (registry of operators and metadata), 2) `Subscription` (tracks an operator release channel like `stable-v1`), 3) `InstallPlan` (dependency resolution engine generating RBAC and CRD installation steps), 4) `ClusterServiceVersion` (CSV, encapsulated metadata detailing container images and CRDs). OLM is natively built into OpenShift.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# subscription-cloudnative-pg.yaml
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: cloudnative-pg-subscription
  namespace: operators
spec:
  channel: stable
  name: cloudnative-pg
  source: operatorhubio-catalog
  sourceNamespace: olm
  installPlanApproval: Automatic # Güvenlik yamalarını otomatik uygula!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Eğer `installPlanApproval: Automatic` seçerseniz, operatör yeni bir ana sürüme (major version) geçtiğinde otomatik güncellenebilir ve bu durum geriye dönük uyumsuzluk yaratabilir. Üretim ortamında `Manual` onay tavsiye edilir.

🇬🇧 WATCH OUT

Setting `installPlanApproval: Automatic` in production can inadvertently trigger breaking major version upgrades. Choose `Manual` approval for enterprise clusters.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- OperatorHub binlerce onaylı ve sertifikalı operatörü tek çatı altında sunar.
- OLM bağımlılıkları çözer, yükseltmeleri yönetir ve operatör sağlığını izler.
- OpenShift Web Konsolunda görsel bir pazar yeri olarak doğrudan entegredir.

- OperatorHub curates thousands of verified, certified community and enterprise operators.
- OLM automates complex operator installations, dependency resolution, and upgrades.
- Exposes a seamless visual marketplace directly inside the OpenShift Web Console.

47

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Java ile Operator Yazımı: Java Operator SDK (JOSDK) Temelleri

Building Operators in Java: Java Operator SDK (JOSDK)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Dünyadaki herkes size 'Kubernetes operatörleri sadece Go (Golang) ile yazılır' diye yalan söyleyebilir. İnanmayın! Bir Java ve Spring Boot geliştiricisi olarak, kendi en iyi bildiğiniz dilde K8s operatörü yazabilirsiniz. Java Operator SDK (JOSDK) sayesinde tek yapmanız gereken bir Java arayüzünü (`Reconciler<SpringBootApp>`) uygulamak ve içine kodunuzu yazmaktır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The tech world might tell you that Kubernetes operators can ONLY be coded in Go. Don't believe the myth! As a Java and Spring Boot engineer, you can build production-grade operators in your favorite language. With Java Operator SDK (JOSDK), you simply implement a clean Java interface (`Reconciler<SpringBootApp>`) and write standard object-oriented code!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Java Operator SDK (JOSDK), Fabric8 Kubernetes Client üzerine kurulmuş modern bir framework'tür. Quarkus veya Spring Boot ile entegre çalışır. `@ControllerConfiguration` anotasyonuyla CRD'ye bağlanır. Reconciler arayüzü tek bir metod içerir: `UpdateControl<SpringBootApp> reconcile(SpringBootApp resource, Context<SpringBootApp> context)`. JOSDK arka planda thread havuzunu, Watch eventlerini, retry mekanizmalarını ve exponential backoff algoritmalarını otomatik yönetir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Java Operator SDK (JOSDK) is a high-performance framework built atop Fabric8 Kubernetes Client, integrating seamlessly with Spring Boot and Quarkus. By implementing `Reconciler<T>` and annotating with `@ControllerConfiguration`, JOSDK manages informers, work queues, event filtering, and exponential backoff retry semantics transparently.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
// Java Operator Reconciler İskeleti:
@ControllerConfiguration(name = "springbootapp-controller")
public class SpringBootAppReconciler implements Reconciler<SpringBootApp> {

    private final KubernetesClient client;

    public SpringBootAppReconciler(KubernetesClient client) {
        this.client = client;
    }

    @Override
    public UpdateControl<SpringBootApp> reconcile(SpringBootApp resource,
Context<SpringBootApp> context) {
        String appName = resource.getMetadata().getName();
        String targetImage = resource.getSpec().getImage();
        int replicas = resource.getSpec().getReplicas();

        // Akıllı Robot Mantığı: Deployment var mı kontrol et, yoksa yarat!
        reconcileDeployment(resource, appName, targetImage, replicas);

        return UpdateControl.noUpdate();
    }
}
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Java ile operatör yazarken Docker imajınızın bellek tüketimine dikkat edin. Standart bir HotSpot JVM operatör için 200MB RAM harcayabilir. GraalVM Native Image derlemesi yaparak operatörünüzü 20MB RAM ile 0.01 saniyede açılan bir ikili dosyaya dönüştürebilirsiniz!

🇬🇧 WATCH OUT

Standard JVM runtime footprints hover around 200MB. For optimal lightweight operator footprints, compile your Java operator with GraalVM Native Image.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Java geliştiricileri Go öğrenmek zorunda kalmadan yerel K8s operatörleri yazabilir.
- Fabric8 client K8s API'sindeki tüm nesneleri tip güvenli (type-safe) Java sınıflarına çevirir.
- Quarkus ve Spring Boot entegrasyonu modern geliştirme konforu sunar.

- Java engineers can build high-performance K8s operators without mastering Go.
- Fabric8 provides type-safe Java POJOs for all native and custom K8s resources.
- Seamless integration with Spring Boot enables dependency injection and metrics.

48

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Pratik Örnek: `SpringBootApp` CRD'sini Yöneten Mini Java Operator

Hands-on: Building a `SpringBootApp` Custom Operator in Java

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kendi canlı robotumuzu sahaya sürme vakti! Bir yazılımcı sadece 5 satırlık minicik bir `SpringBootApp` YAML'ı oluşturacak. Bizim Java operatörümüz bunu anında havada yakalayacak; arka planda otomatik olarak tam teşekküllü bir Deployment, bir Service ve bir Ingress oluşturup ayaklandıracak! Yazılımcı 100 satır karmaşık YAML yazmaktan kurtulacak!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Time to unleash our live robot onto the cluster! A developer submits an ultra-simple 5-line `SpringBootApp` YAML. Our Java Operator intercepts it instantly and automatically generates a full-blown Deployment, ClusterIP Service, and Ingress route under the hood! Developers are spared from writing 150 lines of boilerplate YAML forever!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Operatörün çalışma prensibi: 1) Kullanıcı `kubectl apply -f sample-springbootapp.yaml` ile CR oluşturur. 2) API Server watch eventi tetikler, JOSDK `reconcile` metodunu çağırır. 3) Operatör `client.apps().deployments().inNamespace(...)` ile Deployment'ın varlığını kontrol eder. 4) Yoksa `new DeploymentBuilder()...` akıcı (fluent) API'siyle Deployment oluşturur. 5) `ownerReference` ekleyerek 'sahiplik' kurar; böylece CR silindiğinde açılan tüm deployment ve servisler K8s Garbage Collector tarafından otomatik temizlenir!

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Execution lifecycle: 1) User submits a `SpringBootApp` custom resource. 2) JOSDK dispatches a reconciliation cycle. 3) The reconciler checks for existing child resources via the Fabric8 fluent builder DSL. 4) If absent, it synthesizes matching Deployments and Services. 5) By setting `OwnerReferences`, the operator ensures that deleting the parent CR triggers automated garbage collection of all spawned resources.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# Kullanıcının Yazdığı 5 Satırlık Rüya YAML:
apiVersion: apps.eli5.io/v1alpha1
kind: SpringBootApp
metadata:
  name: my-coffee-shop
  namespace: coffee-shop
spec:
  image: "ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0"
  replicas: 3
  port: 8080
  envMessage: "Operator tarafından yönetilen harika kahve servisi!"

# Operatör bunu yakalar ve arkada Deployment + Service + Ingress'i OTOMATİK kurar!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Alt kaynakları oluştururken mutlaka `OwnerReference` bağlamalısınız! Bağlamazsanız, kullanıcı ana `SpringBootApp` nesnesini sildiğinde arka plandaki Deployment'lar silinmez, kümede 'hayalet iş yükleri' (orphan resources) olarak kalır.

🇬🇧 WATCH OUT

Always attach `OwnerReference` to generated resources! Without owner references, child deployments become orphaned when the custom resource is deleted.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Operatörler karmaşık çoklu kaynak üretimini tek bir basitleştirilmiş CRD'ye indirir.
- `OwnerReference` ile alt kaynakların yaşam döngüsü otomatik bağlanır ve temizlenir.
- Yazılımcı üretkenliği ve platform mühendisliği standartları tavan yapar.

- Operators condense sprawling multi-resource manifests into simple, elegant CRDs.
- OwnerReferences guarantee clean, cascade garbage collection upon deletion.
- Supercharges internal developer platforms (IDP) and engineering team velocity.

BÖLÜM 9 / MODULE 9

FluxCD ile Modern GitOps – Tek Gerçek Kaynak Git!

Modern GitOps with FluxCD – Git as the Single Source of Truth

Canlı sunucuya el sürmeye son: GitOps felsefesi, FluxCD mimarisi, GitHub bootstrap kurulumu, otomatik imaj güncellemesi ve deklaratif Helm yönetimi.

Eliminating manual cluster tinkering: GitOps principles, FluxCD architecture, GitHub bootstrapping, automated image releases, and declarative Helm management.

49

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

GitOps Nedir? Neden `kubectl apply` Artık Yasak?

What is GitOps? Why Manual `kubectl apply` is an Anti-Pattern

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Bir şirkette 10 farklı yazılımcının canlı sunucuya bağlanıp kafasına göre `kubectl apply` yapması, bir restorandaki 10 aşçının aynı çorbaya gizlice farklı tuzlar atmasına benzer (Sonuç tam bir zehirlenme faciası!). Kim ne değiştirdi, dün ne çalışıyordu kimse bilmez! GitOps'ta ise kural kesindir: Canlı sunucuya KİMSE el süremez! Tek gerçek kaynak Git deposudur. Bir şey değişecekse Git'e commit atılır, kümenin içindeki bekçi robot (Flux) değişikliği çeker!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Ten engineers SSH'ing into production to fire off manual `kubectl apply` commands is like ten rogue chefs secretly dumping different spices into the soup without telling each other (Total disaster!). Nobody knows who broke production! GitOps establishes an unbreakable law: Git is the Single Source of Truth. Nobody has cluster write access. You commit changes to Git, and a dedicated cluster agent (Flux) pulls and reconciles them!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Geleneksel CI/CD sistemleri 'Push' (İtme) modeliyle çalışır: Jenkins veya GitLab sunucusu doğrudan K8s üretim kümesinin `cluster-admin` şifresine sahiptir ve dışarıdan kümeye bağlanıp değişiklik basar (Büyük güvenlik riski!). GitOps ise 'Pull' (Çekme) modeliyle çalışır: K8s kümesinin dışarıya hiçbir portu veya yönetim yetkisi açılmaz. Küme içinde çalışan Flux denetleyicisi, dışarıdaki Git deposunu periyodik olarak yoklar (poll) ve kümedeki 'yapılandırma sapmalarını' (configuration drift) otomatik onarır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Traditional CI/CD relies on an insecure 'Push' model: external build runners possess broad `cluster-admin` credentials to inject updates into production. GitOps inverts this via a zero-trust 'Pull' architecture: the cluster exposes no inbound management ports. Instead, an in-cluster agent (Flux) continuously pulls from Git, comparing Git's declared state against live cluster reality and reversing any unauthorized configuration drift.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Geleneksel Tehlikeli Yöntem (Mühendis terminalinden canlıya müdahale):
kubectl apply -f deployment.yaml # YASAK! İz bırakmaz, geri alınamaz!

# GitOps Yolu (Güvenli, Denetlenebilir, Kurumsal Standart):
git add clusters/production/coffee-app/
git commit -m "feat(coffee): upgrade spring-boot to v1.2.0"
git push origin main

# Küme içindeki Flux robotu değişikliği saniyeler içinde otomatik çeker!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

GitOps kurulu bir kümede bir mühendis gidip canlıda `kubectl edit deployment` ile değişiklik yaparsa ne olur? Flux bir dakika içinde bu değişikliği yakalar, 'Git deposunda böyle bir şey yazmıyor!' diyerek değişikliği ezer ve sistemi Git'teki haline geri döndürür!

🇬🇧 WATCH OUT

If a rogue engineer modifies a live pod via `kubectl edit`, Flux detects the drift within 60 seconds, nukes the manual edit, and enforces the exact state declared in Git.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Git tüm altyapının ve yazılımların tek resmi versiyon kütüğüdür.
- Çekme (Pull) modeli sayesinde küme erişim anahtarları dışarıya sızmaz.
- Tüm dağıtım geçmişi `git log` üzerinden kimin ne yaptığıyla açıkça izlenir.

- Git is the immutable, auditable single source of truth for the entire cluster.
- The Pull model eliminates the security hazard of storing cluster tokens in external CI.
- Complete auditability and compliance: every cluster change corresponds to a signed commit.

50

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

FluxCD Mimarisi: 4 Silahşörler Kontrolcüleri

FluxCD Architecture: The Four Musketeer Controllers

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Flux devasa, hantal tek bir program değildir; her biri kendi işinde uzmanlaşmış 4 süper kahraman robottur: 1) Source Controller: Gözlerini kırpmadan Git deposunu ve Docker registry'yi izleyen gözcü. 2) Kustomize Controller: Çekilen YAML'ları K8s kümesine uygulayan usta işçi. 3) Helm Controller: Dışarıdan paket indiren kargo şefi. 4) Notification Controller: Bir şey olduğunda Slack veya Teams'e haber uçuran haberci!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Flux is not a monolithic bloated daemon; it is a modular league of 4 specialized micro-controllers: 1) Source Controller: The tireless scout watching Git repos and OCI registries. 2) Kustomize Controller: The builder reconciling raw manifests into the cluster. 3) Helm Controller: The logistics manager orchestrating Helm chart lifecycles. 4) Notification Controller: The town crier alerting Slack/Teams when events occur!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Flux v2 (GitOps Toolkit) mimarisi Kubernetes Controller Runtime prensiplerine tam uyumludur: 1) `source-controller`: Git depolarını, OCI imajlarını, Helm repo'larını klonlar ve bir gzip tarball olarak küme içi belleğe sunar. 2) `kustomize-controller`: Bu paketi alır, Kustomize yamalarını (overlays) çözer, K8s API'sine uygular ve `prune: true` ile Git'ten silinen kaynakları kümeden de siler. 3) `helm-controller`: Helm paketlerinin sürümlerini ve yükseltmelerini bildirici yönetir. 4) `notification-controller`: Webhook'ları kabul eder.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The Flux GitOps Toolkit is a composable set of specialized controllers: 1) `source-controller` clones Git/Helm/Bucket repositories and serves them as localized tarball artifacts. 2) `kustomize-controller` evaluates manifests via Kustomize engines, applies resources to the API Server, and garbage collects orphaned resources when deleted from Git (`prune: true`). 3) `helm-controller` manages Helm chart releases declaratively. 4) `notification-controller` manages outbound notifications and inbound webhooks.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Flux Kontrolcülerinin Kümedeki Durumunu Sorgula:
flux check

# Çıktı:
# ► checking controllers
# ✔ source-controller: deployment ready
# ✔ kustomize-controller: deployment ready
# ✔ helm-controller: deployment ready
# ✔ notification-controller: deployment ready
# ✔ all checks passed
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Eski Flux v1 tek parçaydı (monolith). Modern Flux v2 ise modüler 'GitOps Toolkit' üzerine kuruludur. Eski v1 dokümanlarındaki `fluxctl` komutlarını kullanmayın; daima modern `flux` CLI'ını kullanın!

🇬🇧 WATCH OUT

Never use legacy Flux v1 tutorials or the deprecated `fluxctl` CLI. Modern Flux v2 is entirely re-architected around the GitOps Toolkit controller family.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Flux modüler yapısı sayesinde sadece ihtiyaç duyulan kontrolcüler kurulabilir.
- Kustomize Controller otomatik çöp toplayıcı (Garbage Collector) gibi çalışır.
- Notification Controller ile dağıtım durumları Slack veya Discord'a bağlanır.

- Micro-controller architecture allows adopting only the controllers you actually need.
- Kustomize controller's `prune: true` automatically purges deleted resources cleanly.
- Notification controller routes release events directly to corporate communication channels.

51

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

Flux Bootstrap: K8s Kümesi ile GitHub Deposunu Nikahlama

Flux Bootstrap: Marrying the K8s Cluster to a GitHub Repo

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Eskiden GitOps kurmak haftalarca süren sertifika ve konfigürasyon eziyetiydi. Flux Bootstrap tek bir sihirli nikah memurudur! Terminale tek bir komut yazarsınız (`flux bootstrap github`): K8s kümenize gider Flux'ı kurar, GitHub hesabınızda yepyeni bir repo açar, içine şifreli SSH anahtarlarını yerleştirir ve ikisini sonsuza dek birbirine bağlar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Setting up GitOps used to require days of manual SSH key juggling. Flux Bootstrap is a one-line automated marriage officiant! You run a single command (`flux bootstrap github`): it installs Flux on your cluster, creates the Git repository on your GitHub account, configures secure deploy keys, and binds them together in holy matrimony!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

`flux bootstrap` komutu hem emirsel başlatıcı hem de bildirici bir döngüdür. Arka planda: 1) GitHub API'sini kullanarak belirtilen organizasyon veya kullanıcı altında Git reposunu oluşturur. 2) Kümede Flux manifestolarını ayağa kaldırır. 3) Bir SSH Deploy Key çifti üretir; genel anahtarı GitHub reposuna 'read/write' yetkisiyle ekler, özel anahtarı ise K8s `flux-system` namespace'ine Secret olarak kilitler. 4) Flux'ın kendi manifestolarını Git'e commit atar; böylece Flux kendi kendini bile GitOps ile günceller!

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The `flux bootstrap` command is idempotent and self-referential: 1) It provisions the target Git repository via the GitHub REST API, 2) Generates a dedicated SSH keypair, installing the public key as a GitHub Deploy Key and the private key as a Kubernetes Secret, 3) Commits its own component manifests into Git (`flux-system`), allowing Flux to upgrade itself via GitOps.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# GitHub Kişisel Erişim Belirteci (Token) Belirle:
export GITHUB_TOKEN="ghp_yourPersonalAccessTokenHere"

# Tek Komutla Flux Bootstrap Kurulumu:
flux bootstrap github \
  --owner=my-org \
  --repository=k8s-gitops-fleet \
  --branch=main \
  --path=clusters/production \
  --personal
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Oluşturduğunuz GitHub Token'ın `repo` izinlerine sahip olduğundan emin olun! Token izinleri yetersizse Bootstrap işlemi reponun açılması veya deploy key eklenmesi aşamasında hata verir.

🇬🇧 WATCH OUT

Your GitHub Personal Access Token requires full `repo` scopes! Insufficient token permissions will fail during deploy key or webhook injection.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Bootstrap işlemi küme ile Git deposu arasındaki güvenli köprüyü kurar.
- Flux kendi kendini yöneten (self-managing) bir yapıya kavuşur.
- Kümenin tüm tanımı Git üzerindeki `clusters/production` klasöründe saklanır.

- Bootstrap establishes the cryptographically secured bridge between Git and cluster.
- Flux manages its own version and configuration via GitOps once bootstrapped.
- Cluster state lives transparently inside the version-controlled `clusters/` directory.

52

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

GitRepository ve Kustomization ile Spring Boot Dağıtımı

Deploying Spring Boot via GitRepository & Kustomization

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Bakkala sipariş verirken iki şeye ihtiyaç vardır: 1) Hangi bakkala gideceğiniz (`GitRepository`: 'Git şu GitHub deposuna bak!'), 2) Bakkaldan ne alacağınız ve eve nasıl yerleştireceğiniz (`Kustomization`: 'Deponun içindeki `coffee-app` klasörünü al, K8s kümesindeki `coffee-shop` ortamına kur ve her 5 dakikada bir kontrol et!').

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Ordering groceries requires two distinct instructions: 1) Where is the grocery store? (`GitRepository`: 'Track this GitHub repository branch!'), 2) What items to fetch and where to put them? (`Kustomization`: 'Extract the manifests inside `./apps/coffee-app`, apply them into the `coffee-shop` namespace, and check every 5 minutes!').

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

1) `GitRepository`: Source Controller'a hangi Git URL'sini, hangi branch'i (`ref.branch: main`) ve hangi kontrol aralığını (`interval: 1m`) izleyeceğini söyler. 2) `Kustomization`: Kustomize Controller'a bu repodaki hangi dizinin (`path: ./clusters/production/coffee-app`) hangi namespace'e uygulanacağını belirtir. `prune: true` parametresiyle Git'ten bir YAML silindiğinde K8s kümesinden de otomatik silinmesi sağlanır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

1) `GitRepository` directs `source-controller` to pull from a Git endpoint, branch, and polling interval, producing an immutable tarball. 2) `Kustomization` instructs `kustomize-controller` to evaluate that tarball at a designated sub-path, target namespace, and interval. Enabling `prune: true` activates automated garbage collection of resources removed from Git.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# spring-boot-gitops.yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: fleet-repo
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/my-org/k8s-gitops-fleet
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: coffee-app-kustomization
  namespace: flux-system
spec:
  interval: 5m
  targetNamespace: coffee-shop
  sourceRef:
    kind: GitRepository
    name: fleet-repo
  path: ./clusters/production/coffee-app
  prune: true
  wait: true # Pod'lar ayağa kalkana kadar dağıtımı bekle!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

`prune: true` parametresi ÇOK GÜÇLÜDÜR! Eğer yanlışlıkla Git deposundan bir YAML dosyasını silip push ederseniz, Flux o servisi ve pod'ları canlı kümeden anında siler! Dikkatli kullanılmalıdır.

🇬🇧 WATCH OUT

`prune: true` is unforgiving! If you delete a manifest file from Git and commit, Flux will immediately delete the live workload from the cluster.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- GitRepository kaynak izleme görevini, Kustomization ise uygulama görevini üstlenir.
- `prune: true` ile yetim kalan (orphaned) eski kaynaklar otomatik temizlenir.
- `flux get kustomizations` komutuyla dağıtım sağlığı anlık olarak izlenir.

- GitRepository handles artifact pulling; Kustomization handles manifest application.
- `prune: true` guarantees complete synchronization by sweeping away orphaned resources.
- Inspect continuous delivery status via `flux get kustomizations`.

53

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

Otomatik İmaj Güncellemesi (ImageUpdateAutomation)

Automated Image Updates: Flux Auto-Commit Magic

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Gözlerinize inanamayacağınız bir otomasyon! GitHub Actions yeni bir Spring Boot Docker imajı derleyip registry'ye yüklediğinde (örn: `v1.0.5`), Flux bunu tespit eder. Ardından Flux KENDİ KENDİNE Git deponuza bir commit atar: 'Ben yeni imajı gördüm, manifestoyu v1.0.5 yaptım!' der, push eder ve küme güncellenir! Hiçbir insana ihtiyaç kalmaz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Pure hands-free engineering magic! When your CI pipeline bakes and pushes a new Spring Boot container image (e.g. `v1.0.5`), Flux sniffs it out. Then Flux ITSELF writes a Git commit to your repository: 'Updated coffee-app image to v1.0.5', pushes the commit, and rolls out the release! Zero human clicks required!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Flux Image Automation üç özel CRD ile çalışır: 1) `ImageRepository`: Docker Registry'yi (GHCR, Quay) tarayarak yeni imaj etiketlerini (tags) keşfeder. 2) `ImagePolicy`: Semantik Sürümleme (SemVer: `1.0.x` veya `^1.0.0`) kurallarına göre en güncel stabil sürümü seçer. 3) `ImageUpdateAutomation`: Git deposundaki YAML dosyasında `# {'$imagepolicy': 'flux-system:coffee-policy'}` işaretçisini bulur, etiketi günceller ve Git'e otomatik yazar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Flux Image Automation orchestrates 3 CRDs: 1) `ImageRepository` scans container registries for new OCI tags. 2) `ImagePolicy` resolves the newest tag using SemVer rules (`range: >=1.0.0 <2.0.0`). 3) `ImageUpdateAutomation` locates setter comments in YAML manifests (`# {"$imagepolicy": ...}`), updates the image tag in-place, and pushes a signed Git commit back to the repo.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# deployment.yaml içindeki Flux Sihirli Yorum Satırı:
spec:
  containers:
    - name: coffee-container
      # Flux bu satırı otomatik güncelleyip Git'e commit atar:
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0 # {"$imagepolicy": "flux-
system:coffee-policy"}

# image-policy.yaml
apiVersion: image.toolkit.fluxcd.io/v1beta2
kind: ImagePolicy
metadata:
  name: coffee-policy
  namespace: flux-system
spec:
  imageRepositoryRef:
    name: coffee-image-repo
  policy:
    semver:
      range: '^1.0.0' # 1.x ailesindeki en yeni yamayı otomatik al!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

ImageUpdateAutomation'ın çalışabilmesi için Flux'ın GitHub deposuna 'YAZMA' (write/push) yetkisinin olması gerekir. Salt-okunur (read-only) deploy key ile bu özellik çalışmaz!

🇬🇧 WATCH OUT

ImageUpdateAutomation requires write permissions to your Git repository! A read-only deploy key will fail when Flux attempts to push automated commits.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Yeni Docker imajları hiçbir manuel müdahale olmadan anında canlıya alınır.
- SemVer politikaları sayesinde sadece test edilmiş stabil yamalar alınır.
- Git deposu her zaman canlıda çalışan gerçek imaj etiketini tam olarak yansıtır.

- Container releases occur automatically without manual manifest editing toil.
- Semantic versioning policies prevent accidental promotion of untested major releases.
- Git commits ensure that your repository always mirrors what is live in production.

54

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

Helm Controller: Helm Paketlerini GitOps ile Yönetme

Declarative Helm Management with Flux HelmReleases

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Helm, Kubernetes dünyasının App Store paket yöneticisidir (npm veya pip gibi). Normalde terminalden `helm install postgresql ...` yazarsınız ama bu GitOps kuralını bozar! Flux'ın `HelmRelease` nesnesiyle deriz ki: 'Bitnami PostgreSQL paketini al, şu şifreleri ver, kur ve güncel tut!' Böylece devasa hazır yazılımları tek bir temiz YAML ile GitOps'a dahil ederiz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Helm is the package manager for Kubernetes (like npm or pip). Normally developers run imperative `helm install postgresql ...` commands, violating GitOps! With Flux's `HelmRelease` resource, we declare: 'Fetch the Bitnami PostgreSQL chart, inject these custom passwords, and keep it healthy!' Sprawling third-party software stacks are tamed cleanly via GitOps!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Helm Controller iki kaynakla yönetilir: 1) `HelmRepository`: Helm chart'larının barındırıldığı depoyu (örn: `https://charts.bitnami.com/bitnami`) tanımlar. 2) `HelmRelease`: Belirli bir chart sürümünü (`version: '15.x'`), değerlerini (`values`) ve yükseltme stratejilerini tanımlar. Flux Helm Controller, chart'ın yükseltmesi başarısız olduğunda otomatik geri alma (`rollback`) yeteneğine sahiptir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Flux manages Helm declaratively via: 1) `HelmRepository` points to Helm chart repositories or OCI registries. 2) `HelmRelease` declares chart versions, target namespaces, and customizable `values` overrides. The Helm Controller triggers native Helm library actions in-cluster, supporting automated rollbacks upon install/upgrade healthcheck failures.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# postgres-helm-release.yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: bitnami
  namespace: flux-system
spec:
  interval: 1h
  url: https://charts.bitnami.com/bitnami
---
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: coffee-postgres
  namespace: coffee-shop
spec:
  interval: 15m
  chart:
    spec:
      chart: postgresql
      version: "15.5.x"
      sourceRef:
        kind: HelmRepository
        name: bitnami
        namespace: flux-system
  values:
    auth:
      database: coffeedb
      username: coffeeuser
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Helm paketlerinin `values` kısmında hassas veritabanı şifrelerini açık metin olarak yazmayın! Şifreler için `valuesFrom` ile bir Secret veya Mozilla SOPS ile şifrelenmiş değerler referans verilmelidir.

🇬🇧 WATCH OUT

Never hardcode cleartext passwords inside `HelmRelease.spec.values`! Reference external Kubernetes secrets using `valuesFrom` or Mozilla SOPS.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- HelmRelease Helm paketlerini tamamen bildirici ve GitOps uyumlu hale getirir.
- Yükseltme hatalarında otomatik rollback yaparak kesintiyi engeller.
- Tüm dış bağımlılıklar (Redis, Kafka, PostgreSQL) Git üzerinden versiyonlanır.

- HelmRelease makes Helm chart lifecycles 100% declarative and GitOps-native.
- Automated rollbacks recover from failed chart upgrades safely.
- Third-party dependencies are cleanly audited and versioned inside your fleet repo.

BÖLÜM 10 / MODULE 10

GitHub Actions CI/CD & Büyük Mezuniyet Senaryosu

GitHub Actions CI/CD & The Grand Enterprise Finale

Uçtan uca otomasyon fabrikası: GitHub Actions iş akışları, Spring Boot Maven testleri, Trivy güvenlik taramaları, GitOps köprüsü ve büyük mezuniyet projesi.

The end-to-end automation factory: GitHub Actions workflows, Spring Boot Maven builds, Trivy CVE scanning, bridging CI to GitOps, and the grand live deployment finale.

55

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

GitHub Actions Temelleri: Workflows, Jobs, Steps ve Runners

GitHub Actions Fundamentals: Workflows, Jobs, Steps, and Runners

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kodunuzu GitHub'a `git push` yaptığınız anda harekete geçen görünmez bir fabrika düşünün! 1) Workflow: Fabrikanın üretim planıdır (`.github/workflows/ci.yaml`). 2) Trigger (Tetikleyici): Zilin çalmasıdır ('Main branch'e kod geldi!'). 3) Jobs (İşler): Fabrikanın farklı departmanlarıdır ('Biri test yapsın, diğeri Docker derlesin'). 4) Runners: Bu işleri yapan bulut bilgisayarlarıdır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imagine an invisible, robotic smart factory that fires up the millisecond you run `git push`! 1) Workflow: The factory blueprint (`.github/workflows/ci.yaml`). 2) Trigger (Event): The starting bell ringing ('Code pushed to main!'). 3) Jobs: Distinct assembly lines ('Line 1 tests Java, Line 2 bakes the container'). 4) Runners: The virtual machines in the cloud executing the steps!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

GitHub Actions mimarisinin yapı taşları: 1) `Events`: İş akışını başlatan olaylar (`push`, `pull_request`, `schedule`, `workflow_dispatch`). 2) `Jobs`: Varsayılan olarak paralel çalışan bağımsız görev bloklarıdır (`needs: [build]` ile sıralı bağımlılık kurulur). 3) `Steps`: Sırayla çalıştırılan tekil kabuk komutları (`run`) veya hazır topluluk eylemleri (`uses: actions/checkout@v4`). 4) `Runners`: Ubuntu, macOS veya Windows barındıran geçici sanal makinelerdir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The core taxonomy of GitHub Actions: 1) `Events` trigger execution (`push`, `pull_request`, webhooks). 2) `Jobs` run concurrently by default on isolated compute runners unless sequenced via `needs: [jobA]`. 3) `Steps` execute sequentially inside a job environment as shell scripts (`run`) or composite action modules (`uses`). 4) `Runners` are ephemeral hosted virtual machines or self-hosted servers.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# .github/workflows/hello-ci.yaml
name: First Spring Boot Workflow
on:
  push:
    branches: [ "main" ]

jobs:
  say-hello:
    runs-on: ubuntu-latest
    steps:
      - name: Kodu Klonla
        uses: actions/checkout@v4

      - name: Terminale Yazdır
        run: |
          echo "🚀 Java Spring Boot CI Pipeline Başlatıldı!"
          echo "Commit SHA: ${{ github.sha }}"
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Bir iş akışındaki adımlar (steps) aynı sanal makinede ardışık çalışır ve dosya sistemini paylaşır. Ancak farklı 'Job'lar farklı sanal makinelerde çalıştığı için aralarında dosya paylaşımı yapmak istiyorsanız `actions/upload-artifact` kullanmalısınız!

🇬🇧 WATCH OUT

Steps within a job share the runner filesystem; distinct Jobs run on separate virtual machines! Pass compiled artifacts between jobs using `upload-artifact` and `download-artifact`.

🎯 Özet Çıkarımlar

- GitHub Actions harici CI sunucusu (Jenkins) kurma ihtiyacını ortadan kaldırır.
- YAML tabanlı bildirici sözdizimi doğrudan projenin Git reposunda saklanır.
- Geniş GitHub Marketplace ekosistemi hazır otomasyon eylemleri sunar.

🎯 Key Takeaways

- Eliminates the maintenance overhead of managing dedicated external CI/CD servers.
- Declarative workflow definitions live directly inside your project repository.
- Leverages a vast marketplace of battle-tested open-source actions for rapid setup.

56

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

Spring Boot CI Pipeline: Maven Test, Build ve Kalite Kontrolü

Spring Boot CI Pipeline: Maven Test, Build & Quality Gates

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Hatalı bir kodun üretim hattına geçmesini engelleyen acımasız kapı bekçisi! Yazılımcı gece yarısı dalgınlıkla bir hataya sebep olduysa veya birim testleri (Unit Tests) bozduysa, GitHub Actions o kodu anında durdurur: Kırmızı çarpı işaretini basar, yöneticilere haber verir ve bozuk kodun Docker imajı olmasına ASLA izin vermez!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The unyielding quality gatekeeper! If a sleepy engineer accidentally introduces a regression or breaks a unit test, GitHub Actions slams the brakes on the assembly line: it raises a red flag, alerts the engineering team, and strictly prevents the broken build from ever producing a container image!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Etkili bir Java CI adımı şunları içerir: 1) `actions/setup-java@v4`: Eclipse Temurin JDK 21 kurulumu ve `cache: 'maven'` ile `~/.m2` deposunun önbelleğe alınması (derleme süresini 4 dakikadan 30 saniyeye düşürür). 2) `mvn clean test`: Tüm JUnit 5 ve Spring Boot MockMvc testlerini çalıştırır. 3) Test raporlarının ve kod kapsama oranlarının (JaCoCo) üretilmesi. 4) `mvn package -DskipTests`: Üretim JAR dosyasının derlenmesi.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A resilient Spring Boot CI pipeline includes: 1) `setup-java` configuring JDK 21 with automatic Maven dependency caching (shrinking build times from minutes to seconds). 2) `mvn clean test` executing JUnit 5 unit and integration test suites. 3) JaCoCo test coverage enforcement. 4) Packaging the final layered production executable JAR via Spring Boot Maven plugin.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Maven CI Job Bloğu:
jobs:
  test-and-package:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: JDK 21 Temurin Kur ve Maven Önbellekle
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: maven

      - name: Birim ve Entegrasyon Testlerini Çalıştır
        run: mvn clean test

      - name: Üretim JAR Paketini Derle
        run: mvn package -DskipTests
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Maven önbellekleme (`cache: maven`) yapmazsanız, GitHub Actions her commit'te tüm Spring Boot kütüphanelerini internetten sıfırdan indirir; bu da dakikalarca zaman kaybına ve GitHub Actions dakikalarınızın hızla tükenmesine yol açar.

🇬🇧 WATCH OUT

Omitting Maven dependency caching forces the runner to re-download hundreds of megabytes of dependencies on every push, wasting pipeline minutes and developer time.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Otomatik testler bozuk kodların müşterilere ulaşmasını %100 engeller.
- Maven önbellekleme derleme sürelerini kat kat hızlandırır.
- Test raporları her Pull Request üzerinde şeffaf biçimde incelenebilir.

- Automated CI testing guarantees that failing code cannot slip unnoticed into production.
- Maven dependency caching slashes build times from minutes down to seconds.
- Test outcomes are surfaced directly on GitHub Pull Requests for peer review.

57

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

Docker İmajı Paketleme, Trivy Güvenlik Taraması ve GHCR Push

Docker Packaging, Trivy Vulnerability Scanning & GHCR Push

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Fabrikadan çıkan Spring Boot uygulamasını çelik bir kargo konteynerine koyuyoruz. Ama hemen gemiye yüklemiyoruz! Önce konteyneri X-Ray cihazından (Trivy güvenlik tarayıcısı) geçiriyoruz. İçinde tehlikeli bir virüs veya güvenlik açığı (CVE) var mı? Temiz raporu çıkarsa, GitHub Container Registry'ye (GHCR) resmi şirket mührüyle teslim ediyoruz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

We pack our freshly compiled Spring Boot binary into a standard steel container. But we don't dispatch it blindly! First, it passes through a high-tech airport X-Ray scanner (Trivy vulnerability scanner). Are there any high-severity CVEs? Only when it receives a clean bill of health do we push it with an immutable cryptographic tag to GHCR!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Bu aşama üç kritik adımdan oluşur: 1) `docker/setup-buildx-action`: Docker Buildx ile hızlı, çok katmanlı önbellek kullanan derleme altyapısı kurar. 2) `aquasecurity/trivy-action`: Üretilen konteyner imajını işletim sistemi paketleri ve Java JAR bağımlılıklarındaki (Log4Shell vb.) bilinen CVE açıkları için tarar; `severity: 'CRITICAL'` bulunursa pipeline'ı keser (`exit-code: 1`). 3) `ghcr.io`: GitHub'ın yerleşik OCI uyumlu konteyner deposuna güvenli push yapılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

This stage executes: 1) Docker Buildx initialization for advanced multi-platform and layer caching. 2) Trivy vulnerability analysis scanning both OS packages and application dependencies against national vulnerability databases; fails the build on Critical CVE thresholds. 3) Authenticated pushing to GitHub Container Registry (`ghcr.io`) using short-lived runner permissions.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Docker Derleme, Tarama ve GHCR Push:
- name: GitHub Container Registry Girişi
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- name: Docker İmajını Derle
  uses: docker/build-push-action@v5
  with:
    context: .
    load: true
    tags: ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}

- name: Trivy ile Güvenlik Taraması Yap
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: 'ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}'
    severity: 'CRITICAL,HIGH'
    exit-code: '0' # Kritik açıkları raporla

- name: İmajı Registry'ye Gönder
  run: docker push ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

İmaj etiketi olarak ASLA `:latest` kullanmayın! `:latest` etiketi hangi kodun canlıda çalıştığını anlamayı imkansız kılar ve K8s imaj önbelleği yüzünden güncellemelerin çekilmemesine neden olur. Her zaman git commit SHA'sını (`:${{ github.sha }}`) kullanın.

🇬🇧 WATCH OUT

NEVER tag images with `:latest` in production pipelines! `:latest` obscures version provenance and defeats Kubernetes image caching. Always tag with the immutable Git commit SHA.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Trivy güvenlik açığı olan paketlerin üretim ortamına sızmasını önler.
- GHCR doğrudan GitHub ekosistemiyle entegre, yüksek hızlı imaj barındırır.
- Her imaj benzersiz Git SHA etiketiyle mühürlenerek tam izlenebilirlik sağlanır.

- Automated Trivy scanning blocks known vulnerable dependencies from reaching production.
- GHCR offers seamless authentication and high-bandwidth image distribution.
- Tagging with Git commit SHA guarantees 100% traceability between code and containers.

58

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

CI ile GitOps Köprüsü: GitHub Actions'ın Flux Deposunu Güncellemesi

Bridging CI to GitOps: GitHub Actions Updating Flux Manifests

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

İki muazzam dünyayı birleştiren altın köprü! GitHub Actions (CI) derlemeyi bitirdiğinde doğrudan gidip K8s kümesine saldırmaz (Bu GitOps kuralına aykırıdır). Bunun yerine gider, Flux'ın izlediği GitOps filosunun deposundaki YAML dosyasını açar: 'İmaj etiketini az önce derlediğim yeni SHA ile güncelledim' diyerek bir commit atar. Ve sahneden çekilir; gerisini Flux halleder!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The Golden Bridge connecting two mighty engineering realms! When GitHub Actions (CI) finishes building, it NEVER attacks the Kubernetes cluster directly (that violates GitOps!). Instead, it walks over to the separate GitOps repository tracked by Flux, updates the image tag inside `deployment.yaml` with the fresh commit SHA, and pushes a clean commit. CI exits; Flux takes the wheel!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kurumsal ayrım prensibi (Separation of Concerns): 1) Kod Reposu (Uygulama kodu + CI pipeline). 2) GitOps Reposu (K8s manifestoları + Ortam ayarları). GitHub Actions derlemeyi bitirdiğinde: `actions/checkout` ile GitOps reposunu klonlar, `clusters/production/.../deployment.yaml` içindeki imaj etiketini yeni derlenen `image: ghcr.io/...:${{ github.sha }}` ile günceller (sed veya Kustomize edit ile), ve GitOps reposuna bir 'chore(release)' commit'i atar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Separation of concerns: 1) App Repo (source code + test pipelines), 2) GitOps Fleet Repo (Kubernetes manifests + infrastructure state). At the tail of the CI run, the workflow clones the fleet repo, updates the target deployment manifest with the newly pushed container SHA via `kustomize edit set image` or `sed`, and commits back to the fleet repo, triggering Flux in-cluster.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# GitHub Actions'tan GitOps Reposuna Otomatik Terfi (Promotion):
- name: GitOps Deposunu Klonla
  uses: actions/checkout@v4
  with:
    repository: 'my-org/k8s-gitops-fleet'
    token: ${{ secrets.GITOPS_DEPLOY_PAT }}
    path: fleet-repo

- name: İmaj Etiketini Güncelle ve Push Et
  run: |
    cd fleet-repo/clusters/production/coffee-app
    sed -i "s|image: .*|image: ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}|
g" deployment.yaml
    git config user.name "github-actions[bot]"
    git config user.email "actions@github.com"
    git add deployment.yaml
    git commit -m "chore(release): promote coffee-app to version ${{ github.sha }}"
    git push origin main
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Uygulama kodunun olduğu repo ile GitOps manifestolarının olduğu repoyu ayrı tutmak en iyi kurumsal pratiktir. Böylece her Java kod değişikliğinde sonsuz bir CI/CD döngüsü (infinite trigger loop) yaşanması engellenir.

🇬🇧 WATCH OUT

Decoupling application repositories from GitOps fleet repositories prevents catastrophic infinite pipeline trigger recursion loops.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- CI altyapısı hiçbir zaman doğrudan Kubernetes kümesine bağlanmaz.
- Dağıtım tetikleyicisi Git üzerindeki temiz bir commit'e dönüşür.
- Tek bir `git revert` komutuyla tüm üretim kümesi önceki sürüme geri döner.

- CI build runners require zero direct network access or credentials to production clusters.
- The release event is formalized as a transparent Git commit in the fleet repo.
- Rolling back production is as simple and safe as executing `git revert` on the fleet repo.

59

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

Çoklu Ortam Dağıtımı (Dev -> Staging -> Prod) ve Canary

Multi-Stage Environment Promotions: Dev, Staging & Production

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Yeni geliştirilen bir ilacı önce laboratuvarda fare üzerinde (Dev ortamı), sonra gönüllü küçük bir grupta (Staging ortamı), her şey yolundaysa tüm dünyada (Production ortamı) kullanıma sunarsınız! Hatta üretimde bile önce müşterilerin sadece %10'una yeni versiyonu gösteren 'Kanarya' (Canary) testi yaparsınız. Bir hata varsa sadece %10 etkilenir, anında geri çekilir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't test a brand-new experimental medicine on the entire global population on day one! You test in the lab (Dev), validate with a small volunteer trial (Staging), and only then release to the public (Production)! And even in production, you use a 'Canary' release: route 10% of users to the new version first. If error rates tick up, kill it instantly before 90% notice!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kustomize 'Overlays' mimarisi ile ortam yönetimi: `base/` (Ortak Deployment ve Service tanımları) üzerine `overlays/dev/` (1 kopya, düşük CPU), `overlays/staging/` ve `overlays/production/` (10 kopya, yüksek CPU, HPA, katı kısıtlar) bindirilir. GitHub Actions 'Environments' mekanizmasıyla Production ortamına geçiş için kıdemli mühendis onayı (Manual Approval Gate) zorunlu kılınabilir. Flagger ile Canary analizleri (HTTP hata oranı <%1) yürütülür.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Multi-tier promotion leverages Kustomize Overlays: a common `base/` manifest stack is patched by `overlays/dev/`, `overlays/staging/`, and `overlays/production/` (varying replicas, resource limits, and secrets). GitHub Environments enforce protection rules requiring manual engineering lead sign-offs before staging can promote to production.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Kustomize Ortam Ağacı:
# apps/coffee-app/
# ├── base/
# │   ├── deployment.yaml
# │   └── service.yaml
# └── overlays/
#     ├── dev/
#     │   └── kustomization.yaml (replicas: 1)
#     └── production/
#         └── kustomization.yaml (replicas: 5, HPA aktif)

# Production Terfisi İçin GitHub Actions Onay Adımı:
# environment:
#   name: production
#   url: https://coffee.production.company.com
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Farklı ortamlar için ayrı ayrı devasa YAML kopyaları oluşturmayın (Kopya-Yapıştır kirliliği!). Ortak şablonu `base/` altında tutun ve sadece değişen ayarları `overlays/` içinde yamalayın (DRY - Don't Repeat Yourself).

🇬🇧 WATCH OUT

Never duplicate massive YAML manifests across environments! Maintain shared definitions in `base/` and patch environment deltas inside `overlays/`.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Kustomize overlays tek bir temel şablonla farklı ortamları yönetir.
- GitHub Actions Onay Kapıları (Approval Gates) kontrolsüz prod geçişlerini durdurur.
- Kanarya (Canary) dağıtımları hata anında felaketi önceden yakalar.

- Kustomize overlays eliminate manifest duplication while supporting environment nuances.
- Manual approval gates prevent unverified releases from hitting production.
- Canary deployments limit blast radius to a microscopic fraction of live traffic.

60

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

BÜYÜK MEZUNİYET FİNALİ: Uçtan Uca Kurumsal Spring Boot Akışı!

THE GRAND FINALE: Full End-to-End Enterprise Flow

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Ve işte büyük orkestranın tüm enstrümanları sahnede! 60 ders boyunca öğrendiğimiz her parça tek bir kusursuz senfoniye dönüşüyor: Bilgisayarınızda Java Spring Boot kodundaki kahve mesajını değiştirip `git push` yapıyorsunuz. GitHub Actions uyanıyor, Maven ile test edip derliyor, Trivy ile tarıyor, Docker imajını GHCR'ye basıyor, GitOps reposunu güncelliyor, kümedeki Flux değişikliği çekiyor, OpenShift rotası trafiği yönlendiriyor ve siz k9s ekranında pod'ların yeşile dönmesini canlı izliyorsunuz! Tebrikler, artık bir Cloud-Native şampiyonusunuz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

AND HERE IS THE GRAND SYMPHONY! All 60 lessons unite into one breathtaking enterprise performance: You edit a single line of Java in Spring Boot and type `git push`. GitHub Actions awakens, runs Maven tests, scans with Trivy, pushes the OCI container to GHCR, commits the release SHA to the GitOps fleet repo, Flux pulls the change in-cluster, OpenShift routes live traffic, and you lean back watching the pods glow neon green in k9s! Congratulations: You are officially a Cloud-Native Master!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Uçtan uca kurumsal yaşam döngüsünün tam özeti: 1) Geliştirici Spring Boot kodunu commitleyip push eder. 2) GitHub Actions CI: `mvn test` -> Docker layered build -> Trivy security scan -> GHCR push. 3) Promotion: Actions, GitOps filosundaki `deployment.yaml` imaj etiketini günceller. 4) Flux GitOps: Değişikliği 30 saniye içinde algılar ve K8s API'sine uygular. 5) K8s/OpenShift: RollingUpdate ile yeni pod'ları ayağa kaldırır, Actuator Startup/Readiness problarını bekler, sıfır kesintiyle trafiği yeni sürüme aktarır. 6) k9s & Prometheus: Sistem mühendisi canlıda logları, CPU/RAM'i ve metrikleri izler.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The definitive Cloud-Native lifecycle synthesis: 1) Dev pushes Java code. 2) GitHub Actions: Maven validation -> Layered Docker build -> Trivy CVE scan -> Push to GHCR. 3) GitOps Promotion: Automation updates fleet manifest with the Git SHA. 4) Flux Reconciliation: Polls Git, detects divergence, applies to K8s API. 5) Orchestration: Zero-downtime RollingUpdate validates Spring Boot Actuator probes before routing traffic. 6) Observability: Real-time verification via k9s HUD and Prometheus telemetry.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# 🚀 BÜYÜK MEZUNİYET KONTROL LİSTESİ:
# 1. Java Kodu: Spring Boot 3.x REST API + Actuator Probes (/actuator/health)
# 2. Docker: Multi-stage build + Layered JAR + Non-Root User (UID 10001)
# 3. K8s: Deployment + ClusterIP + Ingress + HPA + Requests/Limits
# 4. OpenShift: Security Context Constraints (SCC) + Edge Route
# 5. Operators: SpringBootApp CRD + Java Operator SDK Controller
# 6. k9s: Anlık canlı log takibi ('l'), tünel ('shift-f'), kaynak izleme
# 7. FluxCD: GitRepository + Kustomization + ImageUpdateAutomation
# 8. GitHub Actions: Maven Test -> Trivy Scan -> GHCR -> GitOps Auto-Commit

# Tek Komutla Tüm Dünyayı Canlıda İzle:
k9s -n coffee-shop
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Bulut bilişim yolculuğu burada bitmiyor, yeni başlıyor! Gerçek bir mühendis öğrendiği bu 8 teknolojiyi sürekli pratik yaparak, kendi projelerini konteynerize edip GitOps ile yöneterek refleks haline getirir.

🇬🇧 WATCH OUT

The Cloud-Native journey doesn't end here; it begins! Cement this mastery by building, containerizing, breaking, and healing your own microservices continuously.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- 60 derslik tam eğitim: Docker'dan K8s'e, OpenShift'ten Flux ve GitHub Actions'a.
- Tüm akış endüstri standardı Java Spring Boot mikroservis mimarisiyle uygulandı.
- Artık kurumsal bulut-yerel mimarileri baştan sona tasarlama ve yönetme yetkisine sahipsiniz!

- 60 lessons spanning Docker, Kubernetes, OpenShift, k9s, Operators, Flux, and CI/CD.
- Anchored throughout by industry-standard Java Spring Boot production implementations.
- You are fully equipped to design, deploy, and automate enterprise cloud platforms!

🎓 TEBRİKLER! / CONGRATULATIONS!

60 Derslik Büyük Bulut-Yerel Yolculuğu Başarıyla Tamamladınız!

You have successfully conquered all 60 lessons of the Cloud-Native Masterclass!

Docker • Kubernetes • OpenShift • k9s • Operators • FluxCD • GitHub Actions • Java Spring Boot